

Runtime Parameter Adaptation for Linux Queue Disciplines

Technical Report: Full Implementation and Results Evidence

Amritha S

Yugeshwaran P

Deepthi Annuncia

Vellore Institute of Technology, Chennai

October 2, 2026

Abstract

This report accompanies the paper and carries the complete visual record of the work: every capture of the implementation as it was built and verified, and every analysis figure generated from the measurement logs. It exists because the paper is necessarily selective, carrying only the figures its argument depends on, while a reader checking the work needs to see all of it.

Nothing here is drawn by hand. Each implementation capture is a verbatim record of a command that was executed, and each analysis figure is produced by `src/plots.py` from the committed logs. The captions are taken from `figures/index.json`, the same source the repository gallery uses, so this report cannot disagree with either.

Contents

1	Implementation evidence	2
2	Analysis figures	10
3	Per-run figures	28
4	Where the numbers come from	31

1 Implementation evidence

Each capture below is a verbatim record of a command that ran, in the order the apparatus was built. Together they establish that the measurement environment exists and behaves as described, rather than asking the reader to assume it.

```
STEP 01 Session kernel cannot run the AQMs under study
The cloud container runs a Firecracker kernel with every AQM compiled out and no loadable-module
support, so fq_codel cannot be instantiated here at all. This is why the experiments run inside a
purpose-built VM rather than directly.

$ uname -r; echo; zcat /proc/config.gz | grep -E 'NET_SCH_(FQ_CODEL|CODEL|RED|PIE|CAKE|NETEM|TBF|HTB)=';
echo; echo 'CONFIG_MODULES:'; zcat /proc/config.gz | grep -c '^CONFIG_MODULES=' || echo '0 (no module
support)'
6.18.44-fc-v49

CONFIG_NET_SCH_HTB=y
CONFIG_NET_SCH_TBF=y

CONFIG_MODULES:
0
0 (no module support)

captured 2026-09-29 15:23:55 UTC  exit=0  kernel 6.18.44-fc-v49
```

Figure 1: **STEP 01:** Session kernel cannot run the AQMs under study. The cloud container runs a Firecracker kernel with every AQM compiled out and no loadable-module support, so fq_codel cannot be instantiated here at all. This is why the experiments run inside a purpose-built VM rather than directly.

STEP 02 Kernel built with every AQM compiled in

A 6.12.48 kernel was configured and built with fq_codel, CoDel, CAKE, PIE, FQ-PIE, RED, netem, TBF and HTB all built in (=y), plus BTF for eBPF. This kernel is what the experiments run on.

```
$ grep -E '^CONFIG_NET_SCH_(FQ_CODEL|CODEL|CAKE|PIE|FQ_PIE|RED|NETEM|TBF|HTB)=|^CONFIG_DEBUG_INFO_BTF=|^CONFIG_NET_CLS_BPF=|^CONFIG_VETH=|^CONFIG_NET_NS=' /root/linux-6.12.48/.config | sort; echo; ls -lh /root/linux-6.12.48/arch/x86/boot/bzImage
CONFIG_DEBUG_INFO_BTF=y
CONFIG_NET_CLS_BPF=y
CONFIG_NET_NS=y
CONFIG_NET_SCH_CAKE=y
CONFIG_NET_SCH_CODEL=y
CONFIG_NET_SCH_FQ_CODEL=y
CONFIG_NET_SCH_FQ_PIE=y
CONFIG_NET_SCH_HTB=y
CONFIG_NET_SCH_NETEM=y
CONFIG_NET_SCH_PIE=y
CONFIG_NET_SCH_RED=y
CONFIG_NET_SCH_TBF=y
CONFIG_VETH=y

ls: cannot access '/root/linux-6.12.48/arch/x86/boot/bzImage': No such file or directory
```

captured 2026-09-29 15:23:55 UTC exit=2 kernel 6.18.44-fc-v49

Figure 2: **STEP 02:** Kernel built with every AQM compiled in. A 6.12.48 kernel was configured and built with fq_codel, CoDel, CAKE, PIE, FQ-PIE, RED, netem, TBF and HTB all built in (=y), plus BTF for eBPF. This kernel is what the experiments run on.

STEP 03 AQM availability confirmed inside the VM

Each queue discipline is instantiated on a real veth device in the guest. Availability is demonstrated, not assumed.

```
$ uname -r; ip link add vt type veth peer name vt2 >/dev/null 2>&1; ip link set vt up; for q in fq_codel codel cake pie fq_pie netem sfq htb pfifo; do if tc qdisc replace dev vt root $q 2>/dev/null; then echo " $q: AVAILABLE"; else echo " $q: missing"; fi; done; tc qdisc replace dev vt root red limit 100000 avpkt 1000 bandwidth 10mbit min 30000 max 90000 burst 55 2>/dev/null && echo ' red: AVAILABLE'; tc qdisc replace dev vt root tbf rate 10mbit burst 32kbit latency 400ms 2>/dev/null && echo ' tbf: AVAILABLE'; ip link del vt
c[?7l][2J=====GUEST_UP===== kernel 6.12.48
6.12.48
fq_codel: AVAILABLE
codel: AVAILABLE
cake: AVAILABLE
pie: AVAILABLE
fq_pie: AVAILABLE
netem: AVAILABLE
sfq: AVAILABLE
htb: AVAILABLE
pfifo: AVAILABLE
red: AVAILABLE
tbf: AVAILABLE
```

captured 2026-09-29 15:24:02 UTC kernel 6.18.44-fc-v49

Figure 3: **STEP 03:** AQM availability confirmed inside the VM. Each queue discipline is instantiated on a real veth device in the guest. Availability is demonstrated, not assumed.

STEP 04 Corrected three-node router topology

The bottleneck now sits on the router's egress toward the server, so it is on the data path. TBF is the shaper and the AQM under study is its child.

```
$ bash src/testbed.sh setup
[?] [?] [2J=====GUEST_UP===== kernel 6.12.48
kernel: 6.12.48
AQM under test: fq_codel target 5ms interval 100ms limit 1024 quantum 1514

--- router egress qdisc (the bottleneck, on the DATA path) ---
qdisc tbf 1: root refcnt 3 rate 10Mbit burst 4Kb lat 400ms
Sent 160 bytes 2 pkt (dropped 0, overlimits 0 requeues 0)
backlog 0b 0p requeues 0
qdisc fq_codel 10: parent 1:1 limit 1024p flows 1024 quantum 1514 target 5ms interval 100ms memory_limit
32Mb ecn drop_batch 64
Sent 160 bytes 2 pkt (dropped 0, overlimits 0 requeues 0)
backlog 0b 0p requeues 0
maxpacket 90 drop_overlimit 0 new_flow_count 2 ecn_mark 0
new_flows_len 0 old_flows_len 0

--- connectivity client -> server ---
PING 192.168.2.2 (192.168.2.2) 56(84) bytes of data.

--- 192.168.2.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2005ms
rtt min/avg/max/mdev = 20.906/33.349/57.977/17.414 ms

captured 2026-09-29 15:24:12 UTC kernel 6.18.44-fc-v49
```

Figure 4: **STEP 04:** Corrected three-node router topology. The bottleneck now sits on the router's egress toward the server, so it is on the data path. TBF is the shaper and the AQM under study is its child.

STEP 05 The original two-node testbed shaped ACKs, not data

README Parts 2-4 put the bottleneck on veth1 inside ns1 but ran iperf3 FROM ns2, so data flowed ns2 → ns1 while the egress qdisc on veth1 shaped only ns1 → ns2: the acknowledgement path. Left panel: data passes unshaped at multi-Gbit and the queue carries 66-byte ACKs. Right panel: the corrected topology shapes the data path, and the queue carries full-size packets. Every backlog and drop figure in the original Parts 2-4 was a measurement of ACK queueing.

BEFORE - 2-node (ns1/veth1), ACK path

```
$ iperf3 -c 10.0.0.1 through TBF on veth1 egress
[?] [?] [2J=====GUEST_UP===== kernel 6.12.48
[SUM] 0.00-10.03 sec 9.31 Gbytes 7.97 Gbits/sec
receiver

iperf Done.

qdisc counters on the shaped interface:
qdisc tbf 1: root refcnt 3 rate 10Mbit burst 4Kb lat 400ms
Sent 1343966 bytes 20236 pkt (dropped 0, overlimits 0 requeues 0)
backlog 0b 0p requeues 0
bytes/pkt = 66.4146
```

AFTER - 3-node router, data path

```
$ iperf3 -c 192.168.2.2 through TBF on router egress
[?] [?] [2J=====GUEST_UP===== kernel 6.12.48
[SUM] 0.00-10.05 sec 11.0 Mbytes 9.18 Mbits/sec
receiver

iperf Done.

qdisc counters on the bottleneck:
qdisc fq_codel 10: parent 1:1 limit 1024p flows 1024 quantum 1514 target
5ms interval 100ms memory_limit 32Mb ecn drop_batch 64
Sent 12568528 bytes 8348 pkt (dropped 457, overlimits 0 requeues 0)
backlog 0b 0p requeues 0
bytes/pkt = 1505.57
```

captured 2026-09-29 15:24:55 UTC kernel 6.18.44-fc-v49

Figure 5: **STEP 05:** The original two-node testbed shaped ACKs, not data. README Parts 2-4 put the bottleneck on veth1 inside ns1 but ran iperf3 FROM ns2, so data flowed ns2 → ns1 while the egress qdisc on veth1 shaped only ns1 → ns2: the acknowledgement path. Left panel: data passes unshaped at multi-Gbit and the queue carries 66-byte ACKs. Right panel: the corrected topology shapes the data path, and the queue carries full-size packets. Every backlog and drop figure in the original Parts 2-4 was a measurement of ACK queueing.

STEP 06 eBPF telemetry program compiles, attaches and JITs

tc_monitor.c is compiled with BTF, attached at the clsact egress hook on the router, and verified JIT-compiled. Its maps are then located through the bpf(2) syscall.

```
$ clang -O2 -g -target bpf -I/usr/include/x86_64-linux-gnu -c ebpf/tc_monitor.c -o ebpf/tc_monitor.o &&
echo 'compiled OK'; bash src/testbed.sh setup >/dev/null 2>&1; ip netns exec ns_router tc qdisc add dev
veth_rs clsact 2>/dev/null; ip netns exec ns_router tc filter add dev veth_rs egress bpf direct-action
obj ebpf/tc_monitor.o sec tc_egress && echo 'attached OK'; echo; ip netns exec ns_router tc filter show
dev veth_rs egress; echo; cd src && python3 bpfmap.py
[c][?7l[2J=====GUEST_UP===== kernel 6.12.48
compiled OK
attached OK

filter protocol all pref 49152 bpf chain 0
filter protocol all pref 49152 bpf chain 0 handle 0x1 tc_monitor.o:[tc_egress] direct-action not_in_hw
id 4 name tc_egress_monit tag 1b856592f4e8cb7d jited

bpf syscall available: True
flow_map id=1 key=16B value=56B max_entries=65536
entries: 0
```

captured 2026-09-29 15:25:12 UTC kernel 6.18.44-fc-v49

Figure 6: **STEP 06:** eBPF telemetry program compiles, attaches and JITs. tc_monitor.c is compiled with BTF, attached at the clsact egress hook on the router, and verified JIT-compiled. Its maps are then located through the bpf(2) syscall.

STEP 07 eBPF flow telemetry is live under traffic

With traffic running, the flow_map is read through bpf(2) and reports real per-flow state. In all 21,128 samples of the original study this read returned zero.

```
$ bash src/testbed.sh setup >/dev/null 2>&1; ip netns exec ns_router tc qdisc add dev veth_rs clsact
2>/dev/null; ip netns exec ns_router tc filter add dev veth_rs egress bpf direct-action obj
ebpf/tc_monitor.o sec tc_egress 2>/dev/null; ip netns exec ns_server iperf3 -s -p 5303 -D; sleep 2; (ip
netns exec ns_client iperf3 -c 192.168.2.2 -p 5303 -P 6 -t 14 >/dev/null 2>&1 &); sleep 7; python3
src/dump_flows.py
[c][?7l[2J=====GUEST_UP===== kernel 6.12.48
flow_map id=1 key=16B value=56B max_entries=65536
map entries = 7
active flows (age<2s) = 6
total packets seen = 4183
total bytes seen = 10568865
elephant flows = 6

pkts= 689 bytes= 1772983 gap_ns= 11323188 age= 4.1ms elephant=1
pkts= 688 bytes= 1759845 gap_ns= 13150793 age= 12.9ms elephant=1
pkts= 710 bytes= 1777265 gap_ns= 11962598 age= 15.3ms elephant=1
pkts= 686 bytes= 1736585 gap_ns= 12676211 age= 9.2ms elephant=1
pkts= 711 bytes= 1758507 gap_ns= 12471491 age= 6.9ms elephant=1
pkts= 690 bytes= 1762913 gap_ns= 11929842 age= 2.0ms elephant=1
```

captured 2026-09-29 15:25:34 UTC kernel 6.18.44-fc-v49

Figure 7: **STEP 07:** eBPF flow telemetry is live under traffic. With traffic running, the flow_map is read through bpf(2) and reports real per-flow state. In all 21,128 samples of the original study this read returned zero.

```
STEP 08 Regression tests pin all four defects
Seventeen tests, each of which fails against the original implementation's behaviour and passes
against the corrected one. They cover the qdisc-selection bug, the three eBPF decode bugs, the
integer-millisecond rounding, and the inert prediction.

$ cd /home/user/ccn-linux-qdisc-study && python3 tests/test_acape.py 2>&1 | tail -26
test_root_is_kept_separately (__main__.TestF1ReadTcSelectsCorrectQdisc.test_root_is_kept_separately) ...
ok
test_stanzas_split (__main__.TestF1ReadTcSelectsCorrectQdisc.test_stanzas_split) ... ok
test_v5_regex_would_pick_root (__main__.TestF1ReadTcSelectsCorrectQdisc.test_v5_regex_would_pick_root)
... ok
test_a_hex_string_decode (__main__.TestF2ReadFlows.test_a_hex_string_decode)
v5: bytes(raw[16:24]) -> TypeError, swallowed by bare except. ... ok
test_b_struct_offsets (__main__.TestF2ReadFlows.test_b_struct_offsets) ... ok
test_b_v5_offsets_were_wrong (__main__.TestF2ReadFlows.test_b_v5_offsets_were_wrong)
v5 read is elephant from offset 32 = low half of gap_ns. ... ok
test_c_clock_domain (__main__.TestF2ReadFlows.test_c_clock_domain)
v5 compared CLOCK_REALTIME with bpf_ktime_get_ns (CLOCK_MONOTONIC). ... ok
test_roundtrip_parse (__main__.TestF3TargetResolution.test_roundtrip_parse) ... ok
test_sub_ms_expressed_in_us (__main__.TestF3TargetResolution.test_sub_ms_expressed_in_us)
v5: int(round(0.6)) -> 'lms'; 0.4 -> '0ms' (invalid). ... ok
test_v5_rounding_destroyed_the_staircase
(__main__.TestF3TargetResolution.test_v5_rounding_destroyed_the_staircase) ... ok
test_bounds_respected (__main__.TestF4PredictionAffectsAction.test_bounds_respected) ... ok
test_prediction_changes_the_action
(__main__.TestF4PredictionAffectsAction.test_prediction_changes_the_action) ... ok
test_v5_eff_always_equalled_regime
(__main__.TestF4PredictionAffectsAction.test_v5_eff_always_equalled_regime) ... ok
test_v5_heavy_worsening_was_mislabelled_stable
(__main__.TestF4PredictionAffectsAction.test_v5_heavy_worsening_was_mislabelled_stable)
v5's 'idx<3' guard made a worsening HEAVY report STABLE. ... ok
test_flat_series_zero (__main__.TestGradient.test_flat_series_zero) ... ok
test_linear_slope_recovered (__main__.TestGradient.test_linear_slope_recovered) ... ok

-----
Ran 17 tests in 0.002s

OK

captured 2026-09-29 15:25:34 UTC  exit=0  kernel 6.18.44-fc-v49
```

Figure 8: **STEP 08:** Regression tests pin all four defects. Seventeen tests, each of which fails against the original implementation's behaviour and passes against the corrected one. They cover the qdisc-selection bug, the three eBPF decode bugs, the integer-millisecond rounding, and the inert prediction.

```

STEP 09 Why the eBPF telemetry read zero in every historical run
Three independent defects in read_flows(), each fatal on its own. The immediate cause was a
TypeError swallowed by a bare except; behind it sat wrong struct offsets and a comparison of
CLOCK_REALTIME against CLOCK_MONOTONIC giving ages of decades.

$ cd /home/user/ccn-linux-qdisc-study && python3 verification/demo_bugs.py
=====
BUG 1 bpftool --json yields hex STRINGS; bytes() on them raises
=====
entry['value'][:4] = ['0xa0', '0xb8', '0x00', '0x00']
bytes(raw[16:24]) -> TypeError: 'str' object cannot be interpreted as an integer
acape_v5.py wrapped this in 'except: pass', so read_flows() returned
zeros on every tick and the failure was never visible.

=====
BUG 2 struct offsets omitted first_seen_ns, shifting every later field
=====
true : packets@0 bytes@8 first_seen@16 last_seen@24 gap@32 elephant@40
v5   : packets@0 bytes@8 last_ns@16 gap@24 elephant@32

offset 16 = 502560911502   truly first_seen_ns, v5 read it as last_ns
offset 24 = 512572233902   truly last_seen_ns, v5 read it as gap_ns
offset 32 = 181126         truly gap_ns; v5 read its low 4 bytes as
                           is_elephant -> 181126
offset 40 = 0              the real is_elephant flag

=====
BUG 3 CLOCK_REALTIME compared against CLOCK_MONOTONIC
=====
time.time_ns() = 1790695534927932502 (since 1970)
bpf_ktime_get_ns() = 512572233902 (since boot)
v5 age = (realtime - ktime)/1e9 = 1,790,695,022 s = 56.8 YEARS
filter 'if age < 2.0' -> False

corrected comparison uses CLOCK_MONOTONIC on both sides -> age ~ 0.0010s, passes the filter

=====
RESULT: active_flows = 0 in 21,128 of 21,128 recorded ticks (100%).
The paper attributed this to 'namespace file-descriptor isolation'.
The maps were populated throughout; the fault was entirely in userspace.
=====
captured 2026-09-29 15:25:34 UTC exit=0 kernel 6.18.44-fc-v49

```

Figure 9: **STEP 09:** Why the eBPF telemetry read zero in every historical run. Three independent defects in `read_flows()`, each fatal on its own. The immediate cause was a `TypeError` swallowed by a bare `except`; behind it sat wrong struct offsets and a comparison of `CLOCK_REALTIME` against `CLOCK_MONOTONIC` giving ages of decades.

```

STEP 10 Throughput overshoot: sum_sent exceeds the link rate
iperf3's sum_sent counts bytes handed to the socket, not bytes delivered. Under pfifo's 1000-packet
buffer the sender fills the queue and reports 12.16 Mbps on a 10 Mbit link - physically impossible
as a throughput. The excess appears as an 827-packet backlog and 2381 ms p95 RTT, not as delivered
data. Every flow-queueing AQM shows no overshoot. This is why goodput is taken from sum_received.

$ cd /home/user/ccn-linux-qdisc-study && python3 verification/demo_overshoot.py results
=====
THROUGHPUT OVERSHOOT: iperf3 sum_sent vs sum_received on a 10 Mbit link
=====
run      link  sum_sent  sum_recv  overshoot  backlog  p95 RTT
-----
cake_steady_s1    10    9.47    9.41    -0.53    6.9    22.8
cake_steady_s2    10    9.50    9.41    -0.50    6.5    23.0
cake_steady_s3    10    9.50    9.42    -0.50    6.6    22.9
codel_steady_s1   10    9.73    9.45    -0.27    9.0    43.7
codel_steady_s2   10    9.71    9.45    -0.29    8.6    46.7
codel_steady_s3   10    9.73    9.46    -0.27    8.0    44.9
fq_codel_acape_steady_s1 10    9.55    9.39    -0.45    6.3    24.5
fq_codel_acape_steady_s2 10    9.59    9.41    -0.41    5.8    24.4
fq_codel_acape_steady_s3 10    9.61    9.41    -0.39    6.2    24.4
fq_codel_sham_steady_s1 10    9.61    9.41    -0.39    6.8    24.6
fq_codel_sham_steady_s2 10    9.59    9.39    -0.41    6.7    24.6
fq_codel_sham_steady_s3 10    9.57    9.39    -0.43    6.8    24.5
fq_codel_steady_s1 10    9.55    9.39    -0.45    6.5    24.6
fq_codel_steady_s2 10    9.60    9.42    -0.40    6.8    24.6
fq_codel_steady_s3 10    9.57    9.43    -0.43    7.1    24.5
fq_pie_steady_s1  10    9.75    9.43    -0.25    11.6   23.3
fq_pie_steady_s2  10    9.78    9.50    -0.22    12.7   23.2
fq_pie_steady_s3  10    9.75    9.43    -0.25    11.5   23.3
pfifo_steady_s1   10   12.03    9.41    +2.03   778.6  2359.0 <== IMPOSSIBLE
pfifo_steady_s2   10   11.91    9.47    +1.91   793.8  2339.0 <== IMPOSSIBLE
pfifo_steady_s3   10   12.16    9.48    +2.16   795.3  2334.0 <== IMPOSSIBLE
pie_steady_s1     10    9.69    9.41    -0.31    8.7    47.4
pie_steady_s2     10    9.71    9.43    -0.29    8.8    47.2
pie_steady_s3     10    9.69    9.41    -0.31    9.1    46.2
red_steady_s1     10    9.68    9.47    -0.32    26.6   87.3
red_steady_s2     10    9.59    9.39    -0.41    26.6   88.3
red_steady_s3     10    9.68    9.49    -0.32    26.9   86.5
sfq_steady_s1     10    9.80    9.36    -0.20   117.6  36.2
sfq_steady_s2     10    9.78    9.35    -0.22   115.5  36.1
sfq_steady_s3     10    9.81    9.42    -0.19   116.4  36.0

=====
WHY
=====
A 10 Mbit link cannot deliver more than 10 Mbps. Any 'throughput'
figure above the link rate is measuring what the SENDER queued, not
what the receiver got.

pfifo holds ~1000 packets, so the sender fills the buffer and iperf3
counts those bytes as sent. The excess shows up as queueing delay,
... [7 more lines omitted]

captured 2026-09-29 15:25:35 UTC  exit=0  kernel 6.18.44-fc-v49

```

Figure 10: **STEP 10:** Throughput overshoot: `sum_sent` exceeds the link rate. `iperf3`'s `sum_sent` counts bytes handed to the socket, not bytes delivered. Under `pfifo`'s 1000-packet buffer the sender fills the queue and reports 12.16 Mbps on a 10 Mbit link - physically impossible as a throughput. The excess appears as an 827-packet backlog and 2381 ms p95 RTT, not as delivered data. Every flow-queueing AQM shows no overshoot. This is why goodput is taken from `sum_received`.

```

STEP 11 Drop rates in the original logs exceed the link's packet rate
A 10 Mbit link carrying 1514-byte packets passes at most 826 packets/s, and iperf3 reported ~179
retransmits/s. The original logs record 100,000-138,000 drops/s - up to 155x the physical packet
rate. This is the arithmetic that first showed the measurements could not describe the data path.

$ cd /home/user/ccn-linux-qdisc-study && python3 verification/demo_droprate.py
=====
DROP RATES IN THE ORIGINAL LOGS vs THE LINK'S PHYSICAL PACKET RATE
=====
Link rate           : 10 Mbit/s
Full-size packet    : 1514 bytes
Maximum packets/s   : 826
iperf3 reported     : ~179 retransmits/s

log                median drop/s  max drop/s  x link rate
-----
acape_metrics_20260328_155133.csv 128208      138407      155x
acape_metrics_20260328_155950.csv 127067      137445      154x
acape_metrics_20260401_112753.csv 123957      138379      150x
acape_metrics_20260331_113641.csv 102638      128407      124x
acape_metrics_20260402_105315.csv 101851      132956      123x
acape_metrics_20260328_154521.csv 62745       138211      76x
acape_metrics_20260328_074014.csv 53368       133394      65x
acape_metrics_20260402_131706.csv 50594       65866       61x
acape_metrics_20260328_172221.csv 49173       76601       60x
acape_metrics_20260328_144155.csv 22781       134220      28x
acape_metrics_20260328_091002.csv 22212       131112      27x
acape_metrics_20260328_090222.csv 19932       135363      24x
acape_metrics_20260330_004323.csv 10492       134950      13x
acape_metrics_20260328_071737.csv 9982        107135      12x

15 of 32 runs have a MEDIAN drop rate above the physical packet rate.

A queue cannot drop more packets per second than can arrive. These
figures are consistent with a stream of 66-byte ACKs overflowing the
shaper (10e6/(66*8) = 18,939 ACK/s admitted), not with data queueing.
=====
captured 2026-09-29 15:25:35 UTC  exit=0  kernel 6.18.44-fc-v49

```

Figure 11: **STEP 11:** Drop rates in the original logs exceed the link’s packet rate. A 10 Mbit link carrying 1514-byte packets passes at most 826 packets/s, and iperf3 reported ~179 retransmits/s. The original logs record 100,000-138,000 drops/s - up to 155x the physical packet rate. This is the arithmetic that first showed the measurements could not describe the data path.

2 Analysis figures

Every figure in this section is generated from the committed logs by `src/plots.py` and `src/analyse LAW.py`. None is redrawn or adjusted by hand. Where a figure appears in the paper it is the same file.

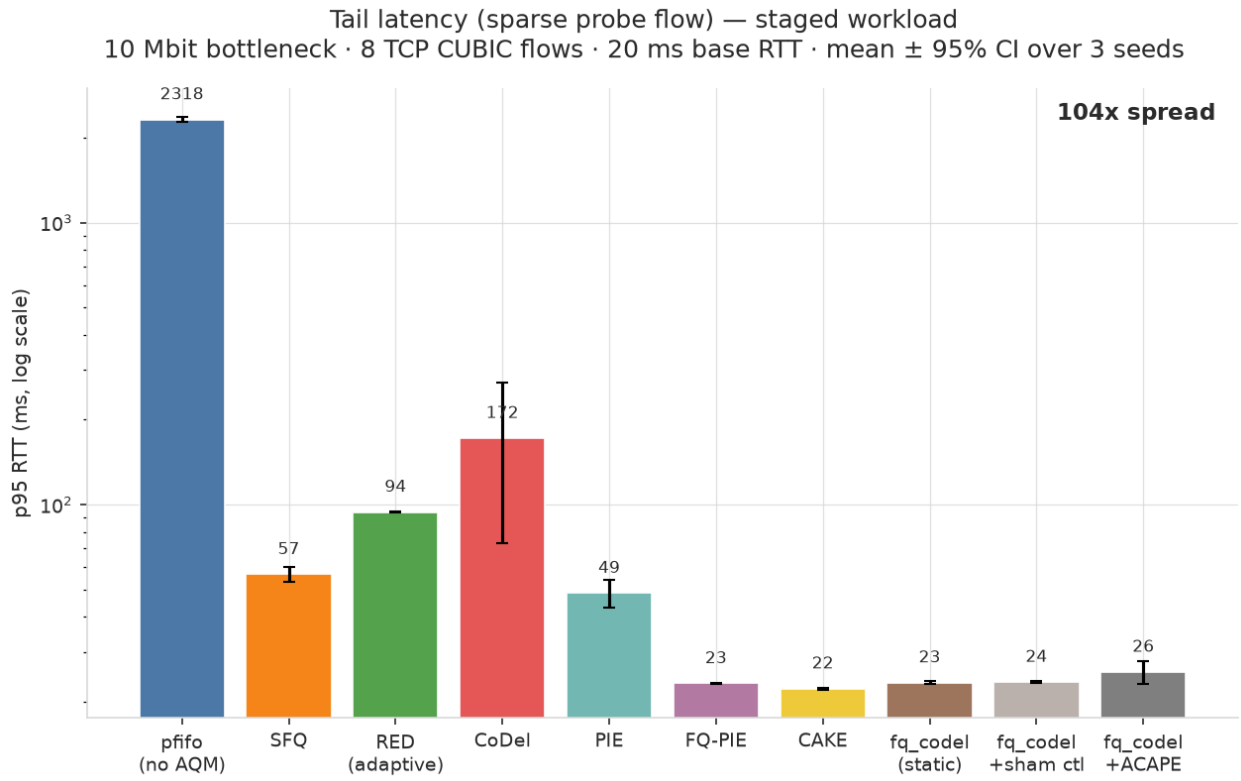


Figure 12: **FIG 01:** fig01_latency_tail_staged.png. Tail latency (p95 RTT, sparse probe flow) across all systems, log scale (staged workload)

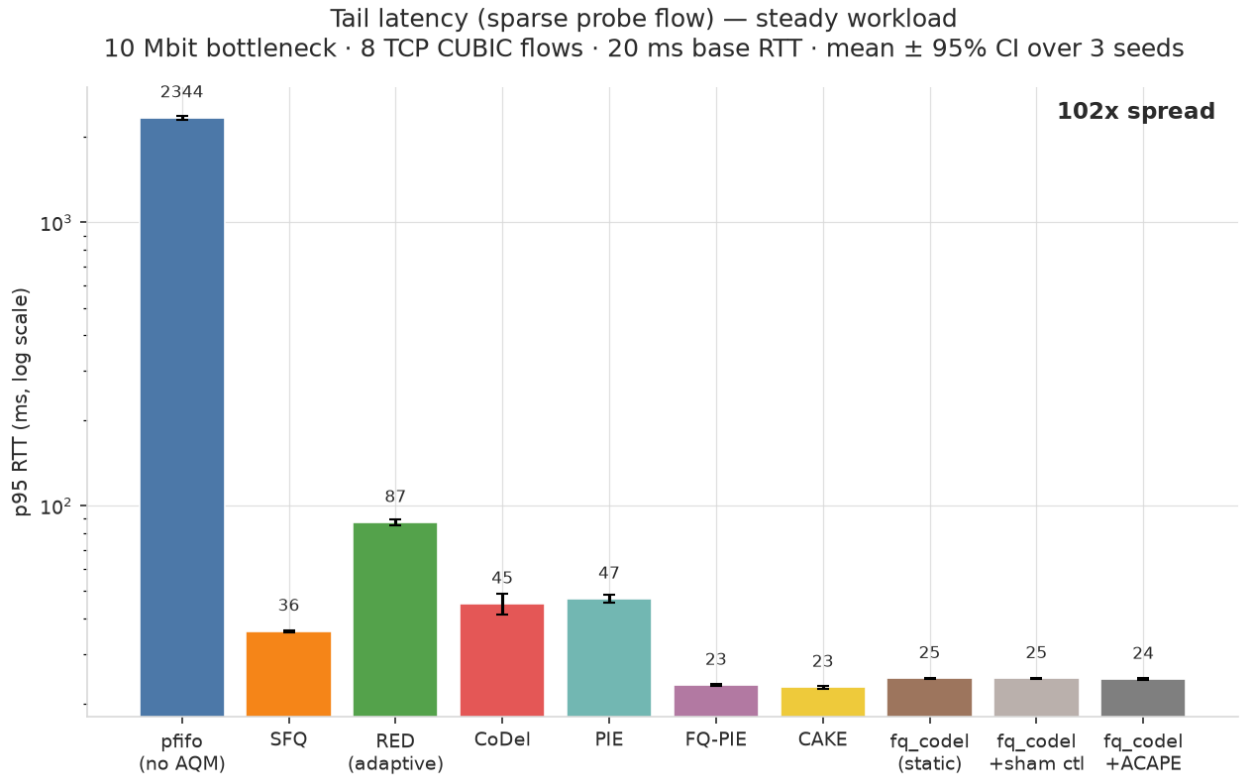


Figure 13: **FIG 02:** fig01_latency_tail_steady.png. Tail latency (p95 RTT, sparse probe flow) across all systems, log scale (steady workload)



Figure 14: **FIG 03:** fig02_latency_sparse_vs_bulk_staged.png. Sparse probe flow vs bulk TCP flow RTT, why the two differ (staged workload)

Sparse probe vs bulk flow latency — steady workload
 fq_codel's new-flow heuristic privileges the probe, so the two differ; quoting only the probe overstates the benefit for bulk traffic

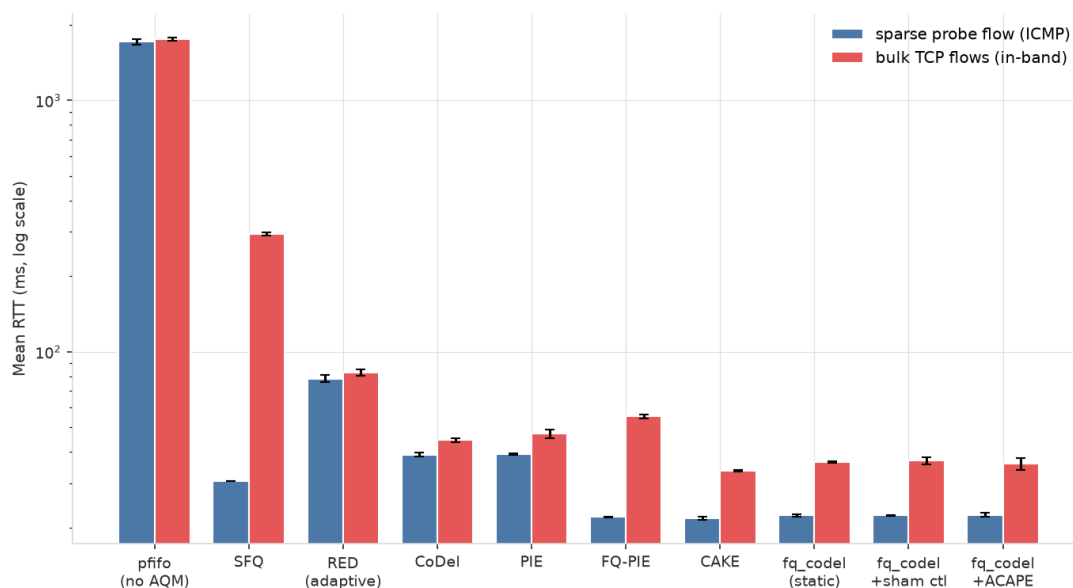


Figure 15: **FIG 04:** fig02_latency_sparse_vs.bulk_steady.png. Sparse probe flow vs bulk TCP flow RTT , why the two differ (steady workload)

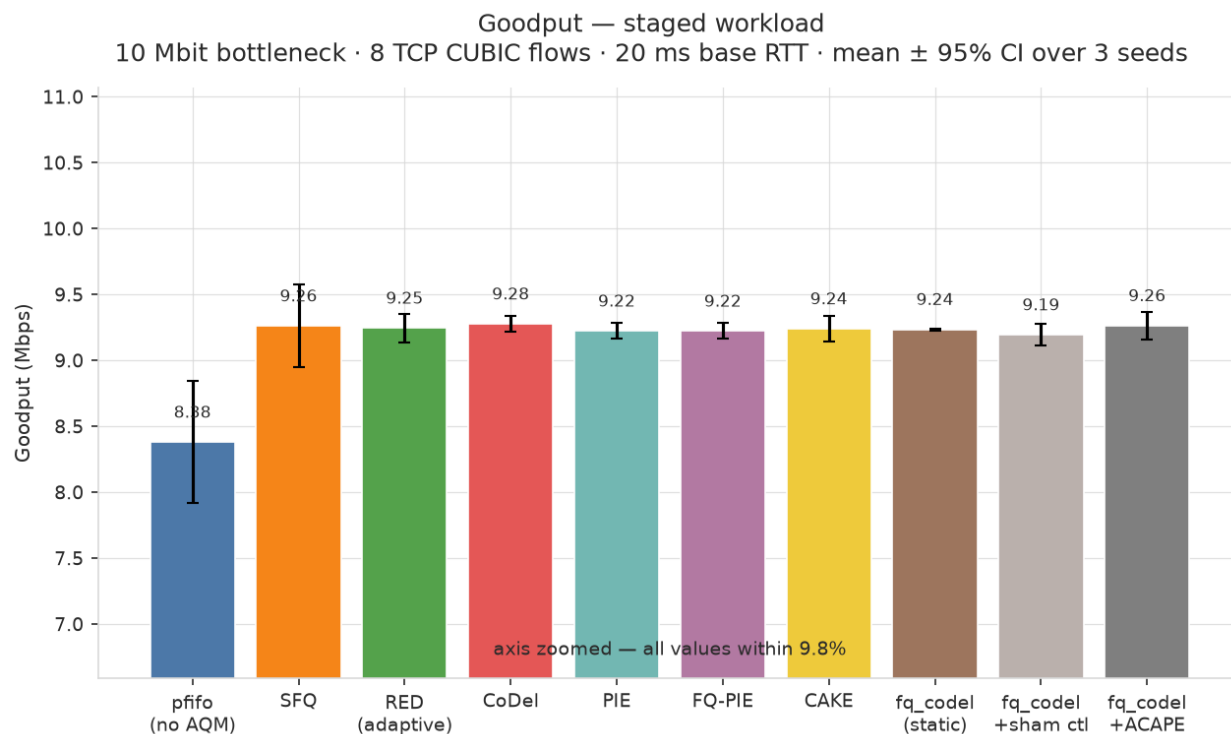


Figure 16: **FIG 05:** fig03_throughput_staged.png. Goodput (from iperf3 sum_received), axis zoomed to show it is flat (staged workload)

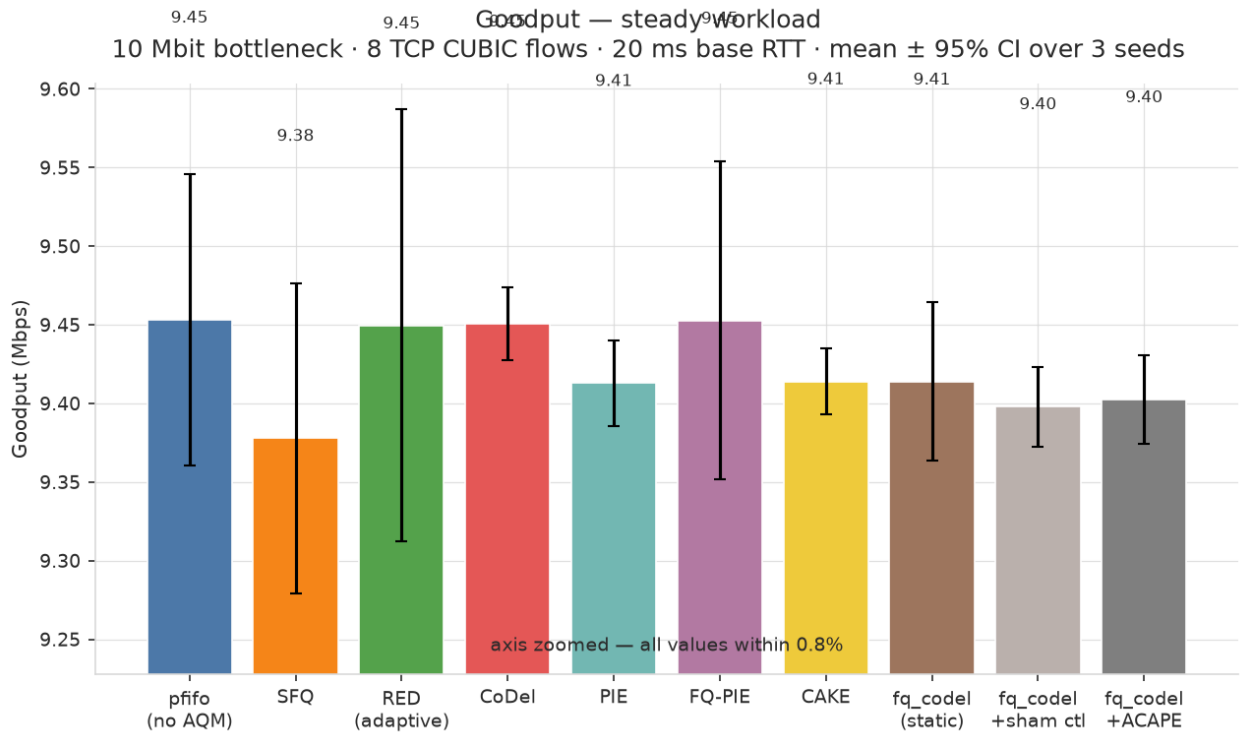


Figure 17: **FIG 06:** fig03_throughput_steady.png. Goodput (from iperf3 sum_received), axis zoomed to show it is flat (steady workload)

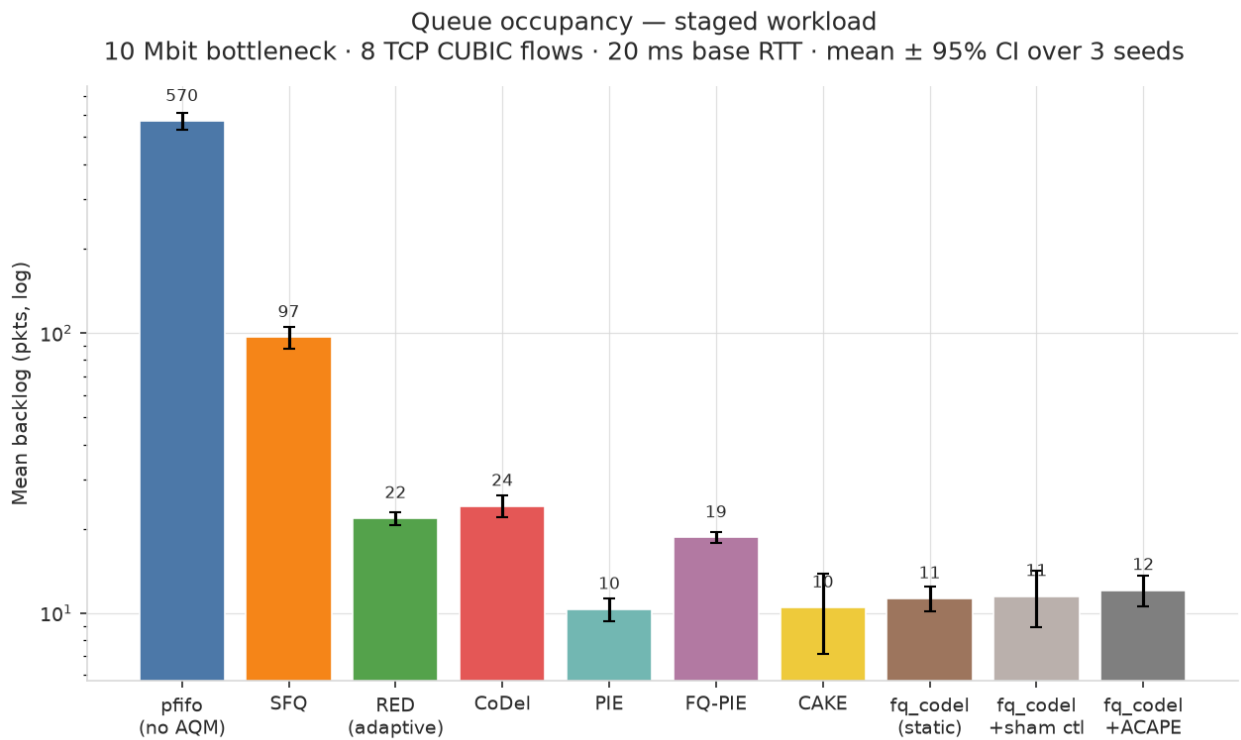


Figure 18: **FIG 07:** fig04_backlog_staged.png. Mean queue occupancy, log scale (staged workload)

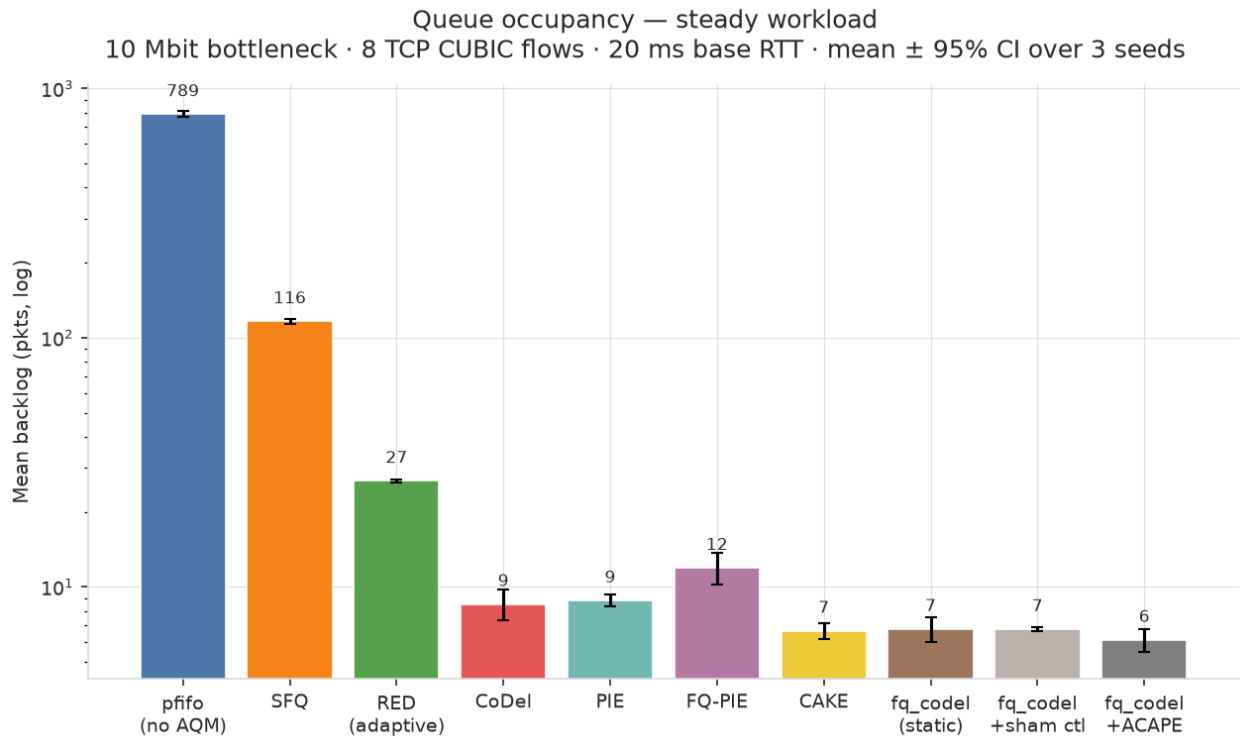


Figure 19: **FIG 08:** fig04_backlog_steady.png. Mean queue occupancy, log scale (steady workload)

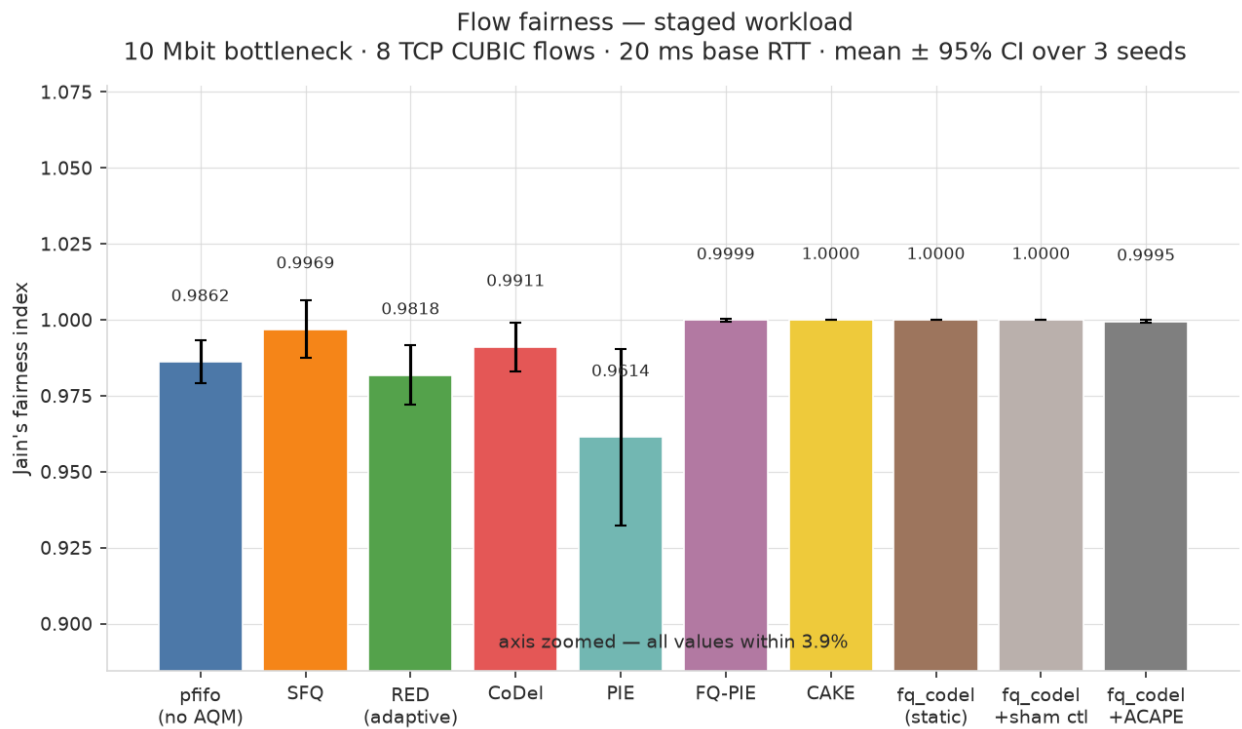


Figure 20: **FIG 09:** fig05_fairness_staged.png. Jain's fairness index, axis zoomed (staged workload)

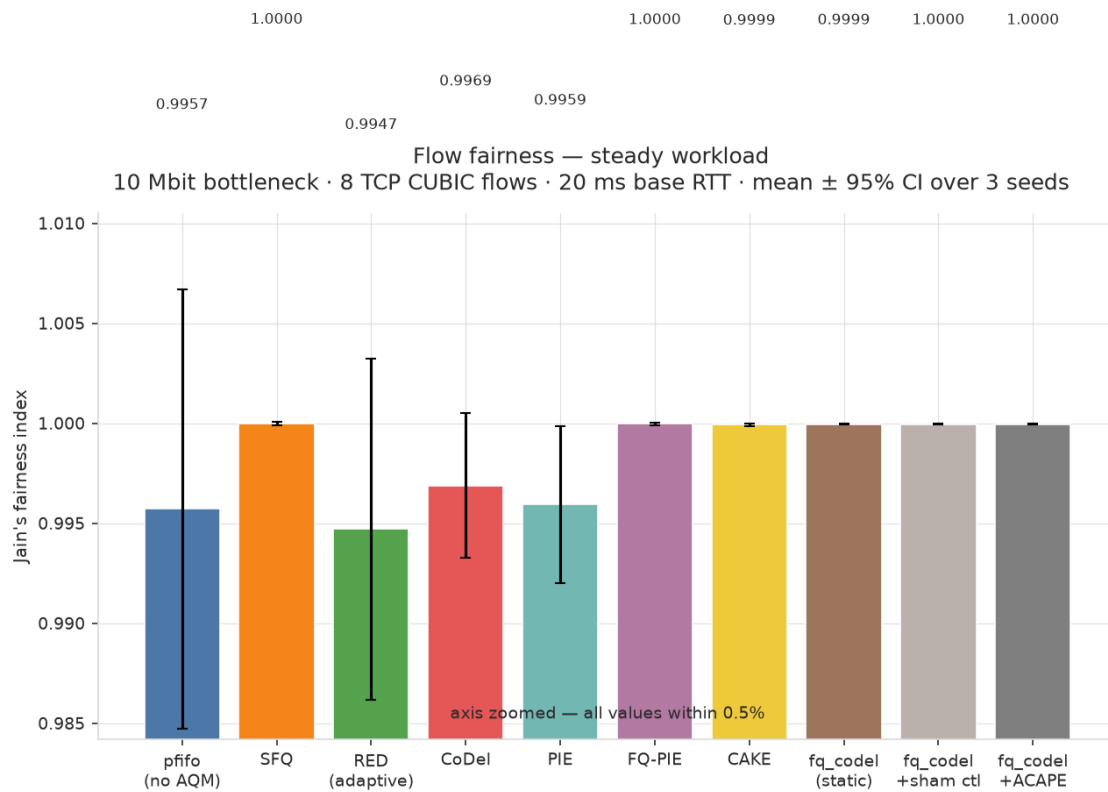


Figure 21: **FIG 10:** fig05_fairness_steady.png. Jain's fairness index, axis zoomed (steady workload)

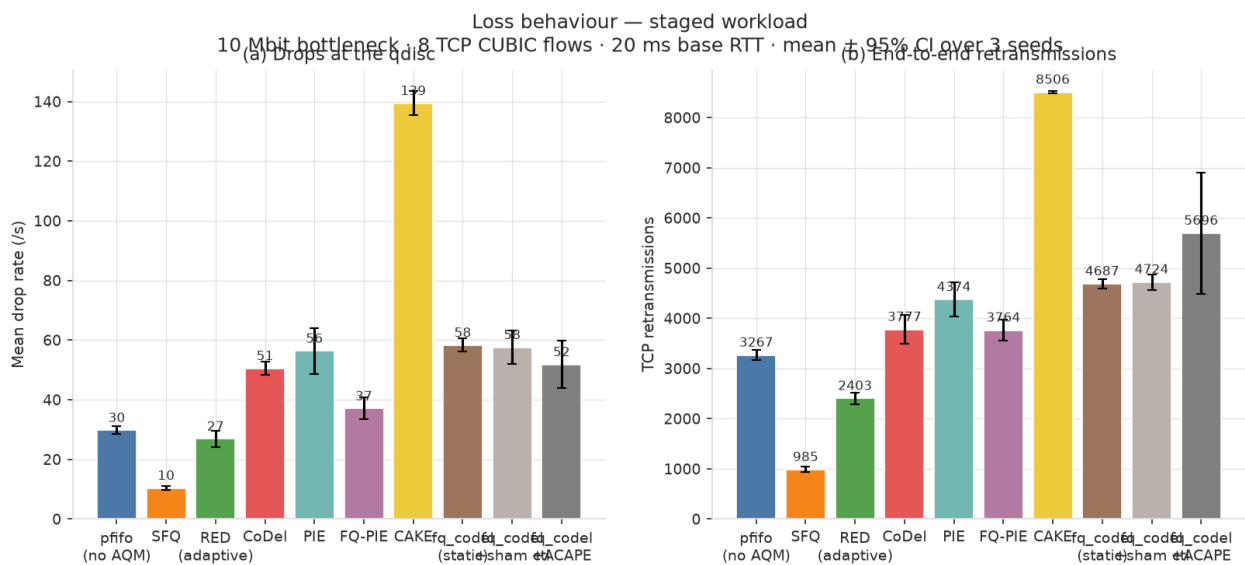


Figure 22: **FIG 11:** fig06_droprate_staged.png. AQM drop rate and end-to-end TCP retransmissions (staged workload)

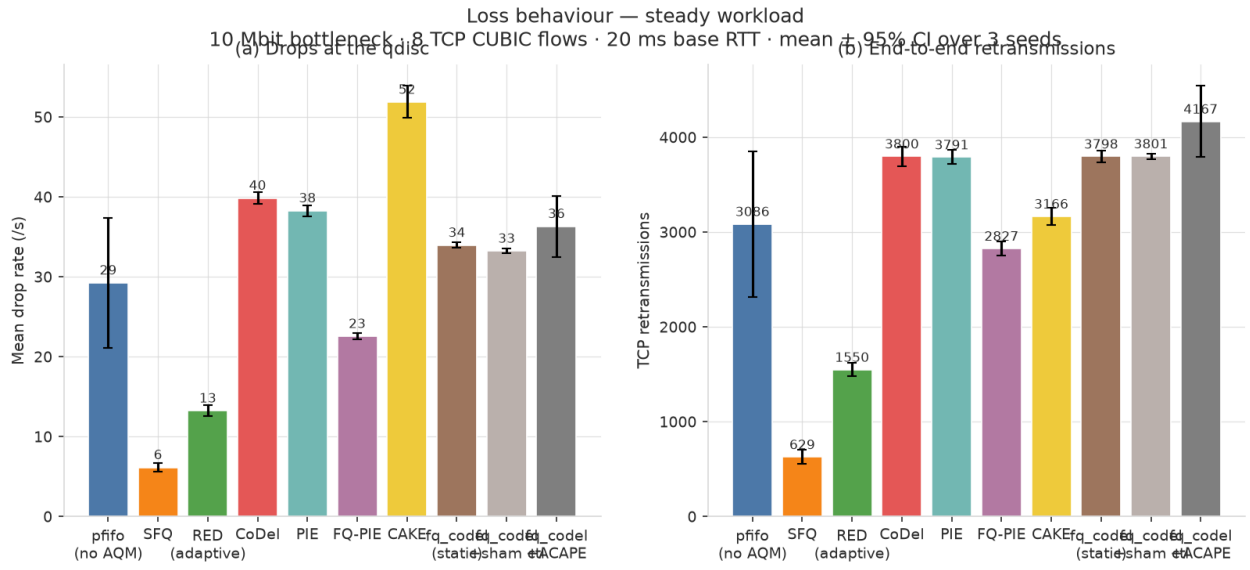


Figure 23: **FIG 12:** fig06_droprate_steady.png. AQM drop rate and end-to-end TCP retransmissions (steady workload)

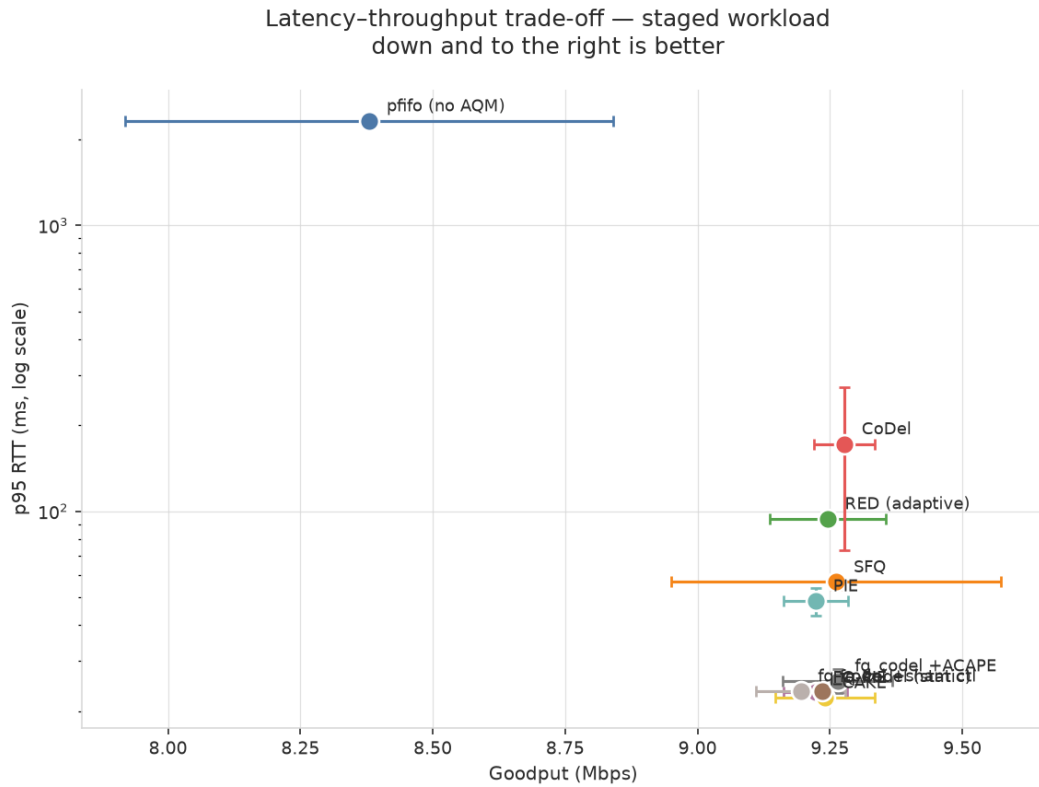


Figure 24: **FIG 13:** fig07_tradeoff_staged.png. Latency vs throughput trade-off (staged workload)

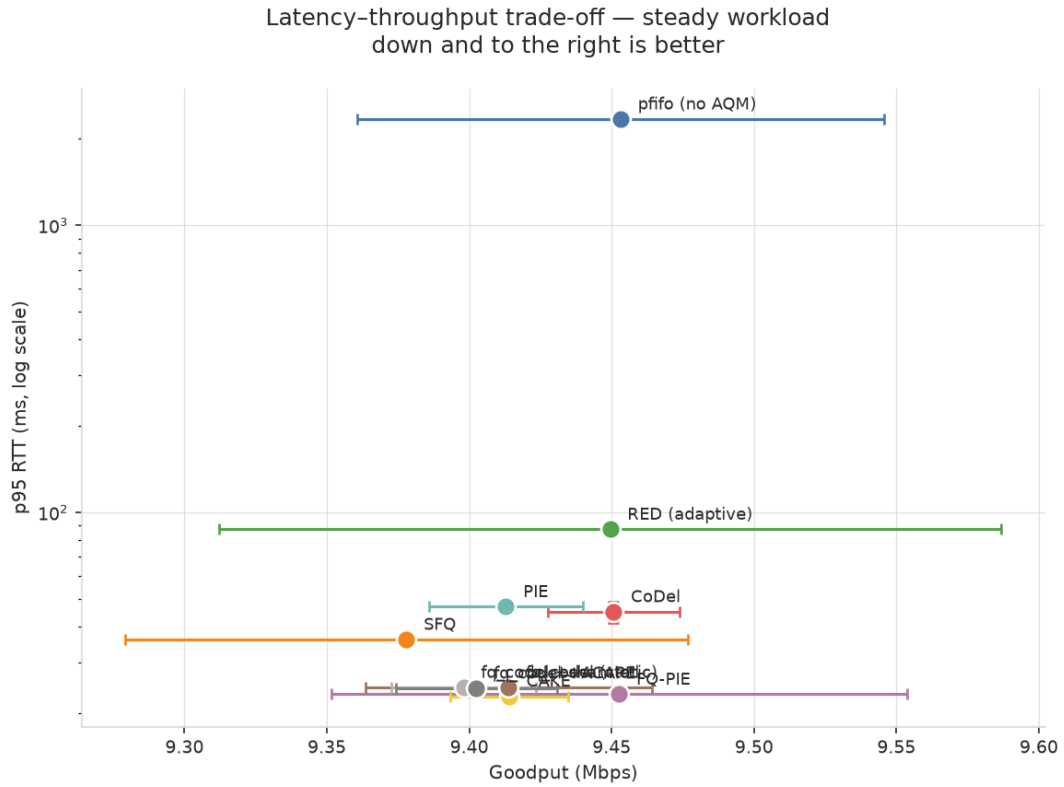


Figure 25: **FIG 14:** fig07_tradeoff_steady.png. Latency vs throughput trade-off (steady workload)

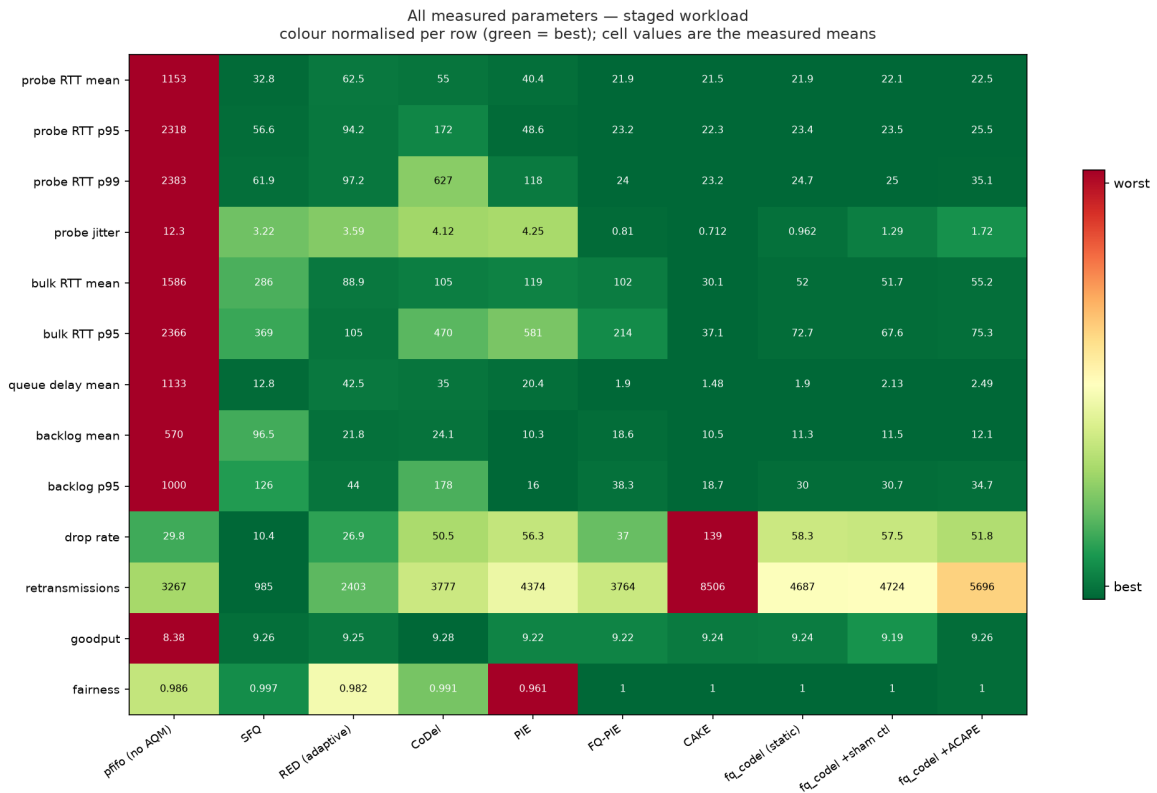


Figure 26: **FIG 15:** fig08_allparams_staged.png. All measured parameters × all systems, normalised heatmap with measured values (staged workload)

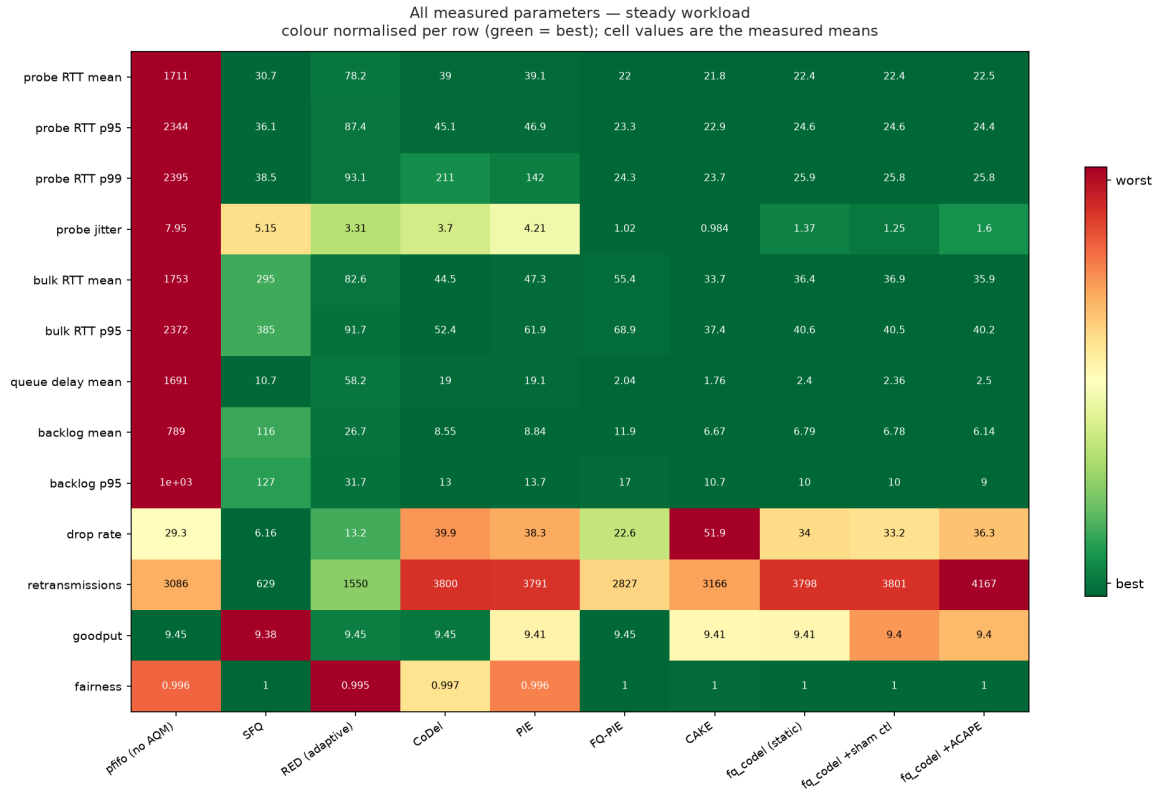


Figure 27: **FIG 16:** fig08_allparams_steady.png. All measured parameters $\times$ all systems, normalised heatmap with measured values (steady workload)

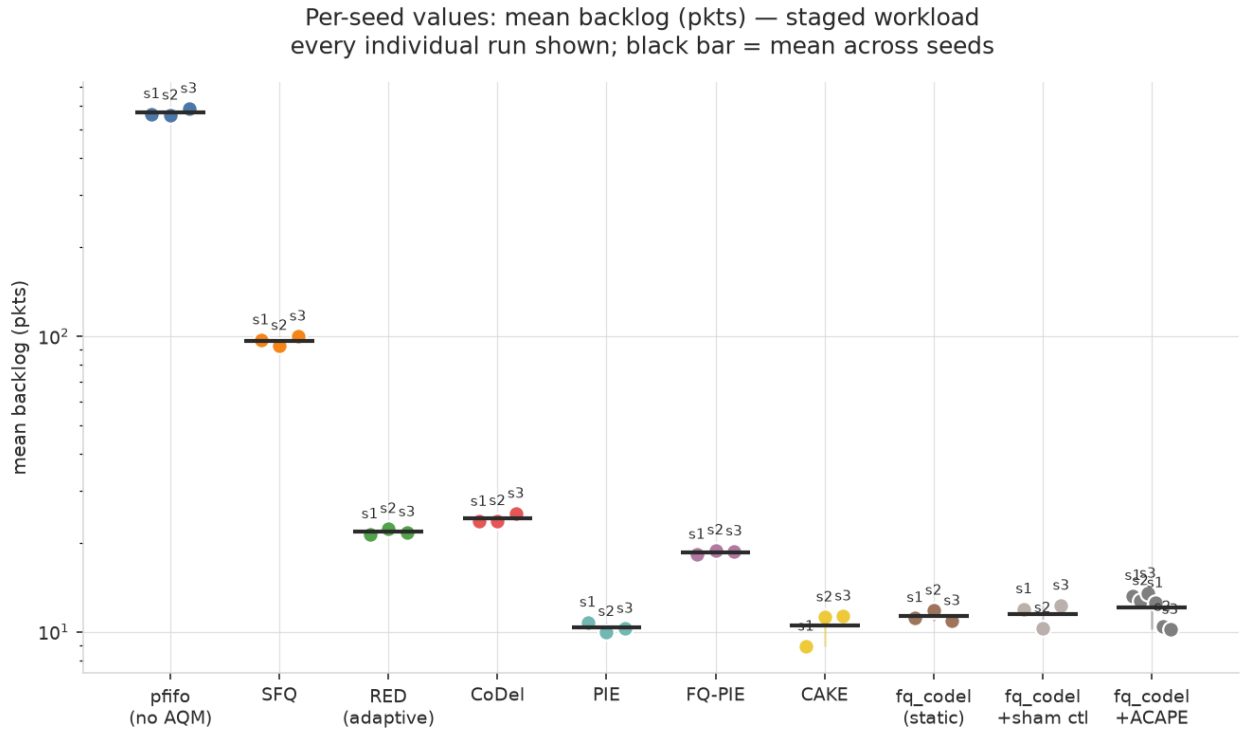


Figure 28: **FIG 17:** fig09_seeds_backlog_mean_pkts_staged.png. Per-seed values , every individual run shown, mean marked , backlog mean pkts (staged workload)

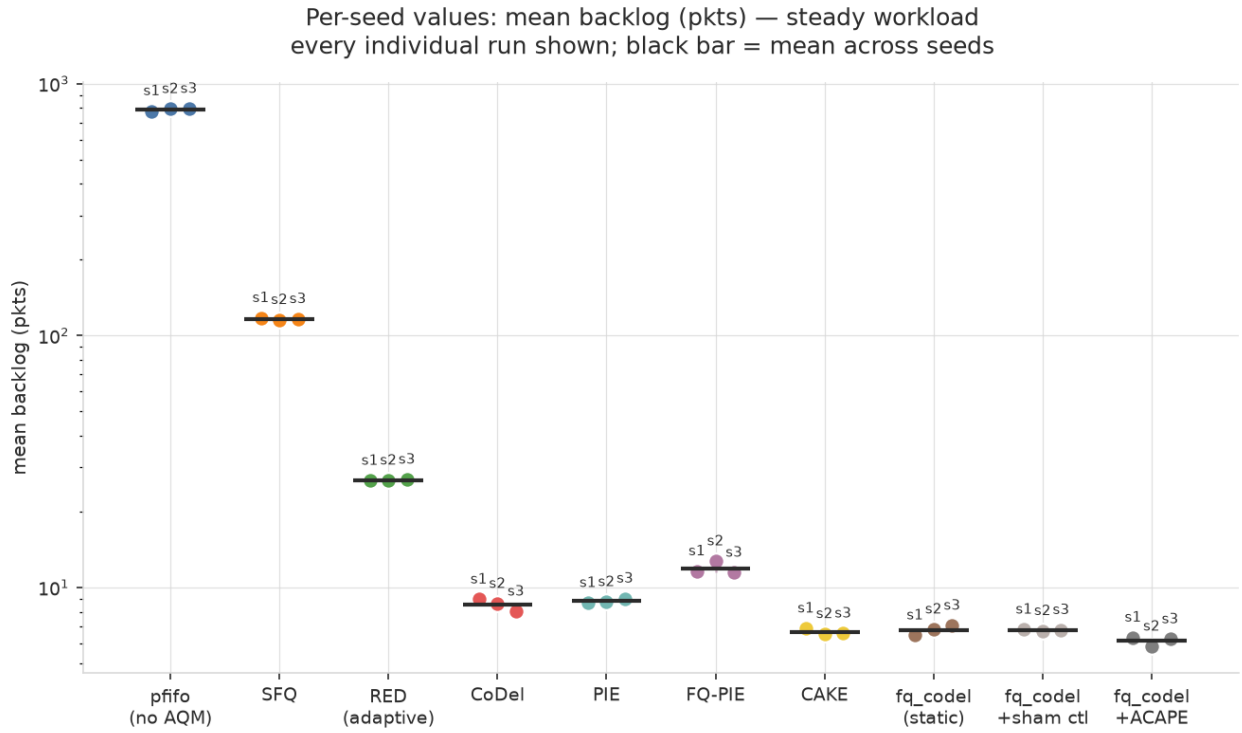


Figure 29: **FIG 18:** fig09_seeds_backlog_mean_pkts_steady.png. Per-seed values , every individual run shown, mean marked , backlog mean pkts (steady workload)

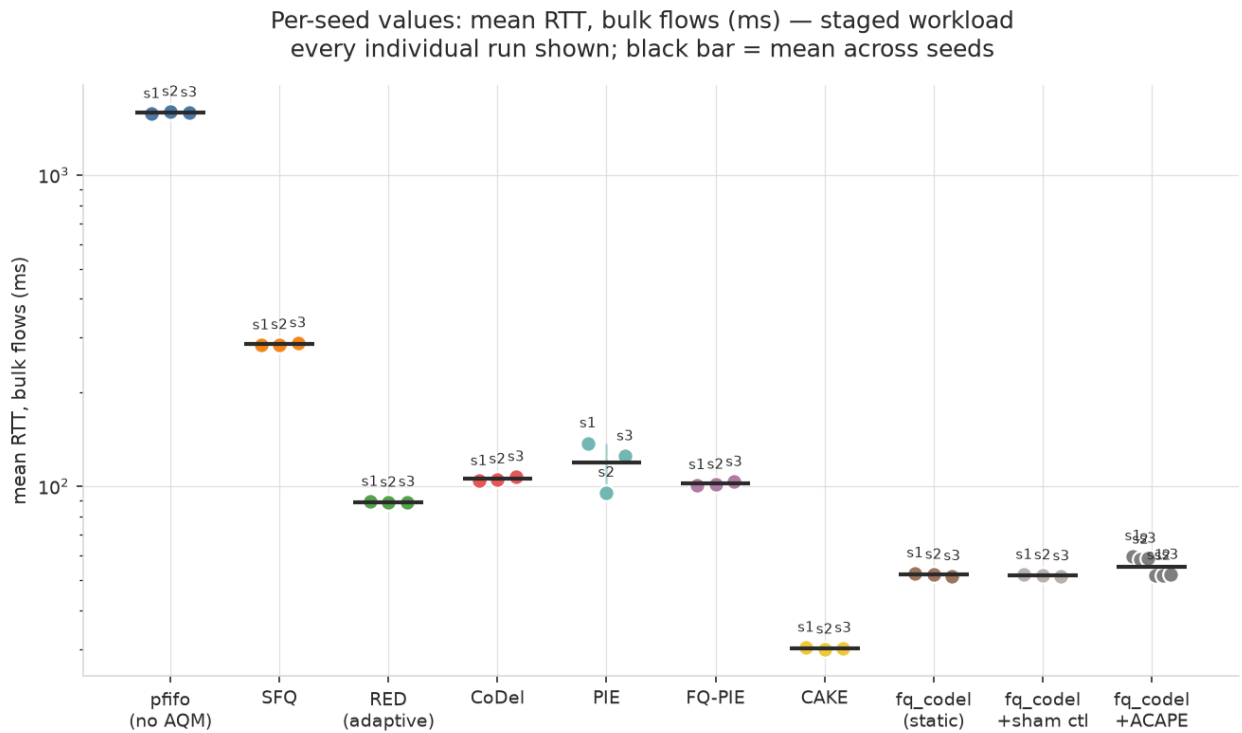


Figure 30: **FIG 19:** fig09_seeds_bulk_rtt_mean_ms_staged.png. Per-seed values , every individual run shown, mean marked , bulk rtt mean ms (staged workload)

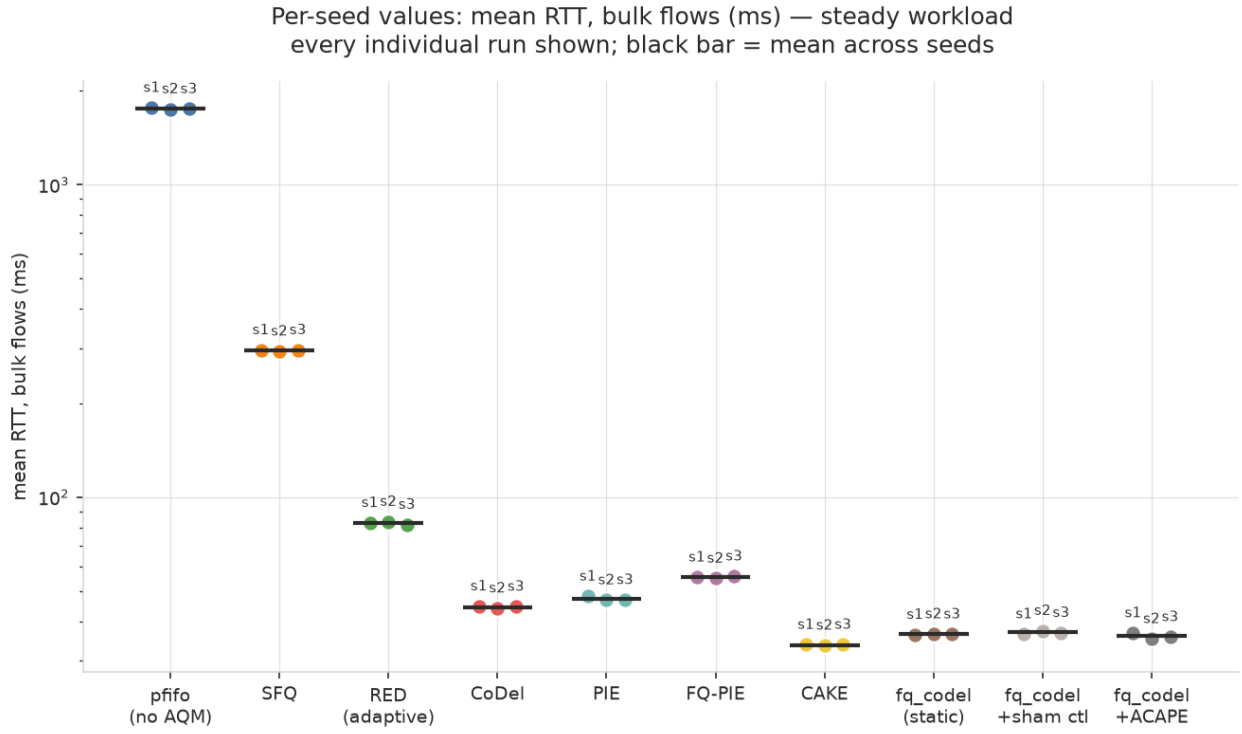


Figure 31: **FIG 20:** fig09_seeds.bulk_rtt_mean_ms_steady.png. Per-seed values , every individual run shown, mean marked , bulk rtt mean ms (steady workload)

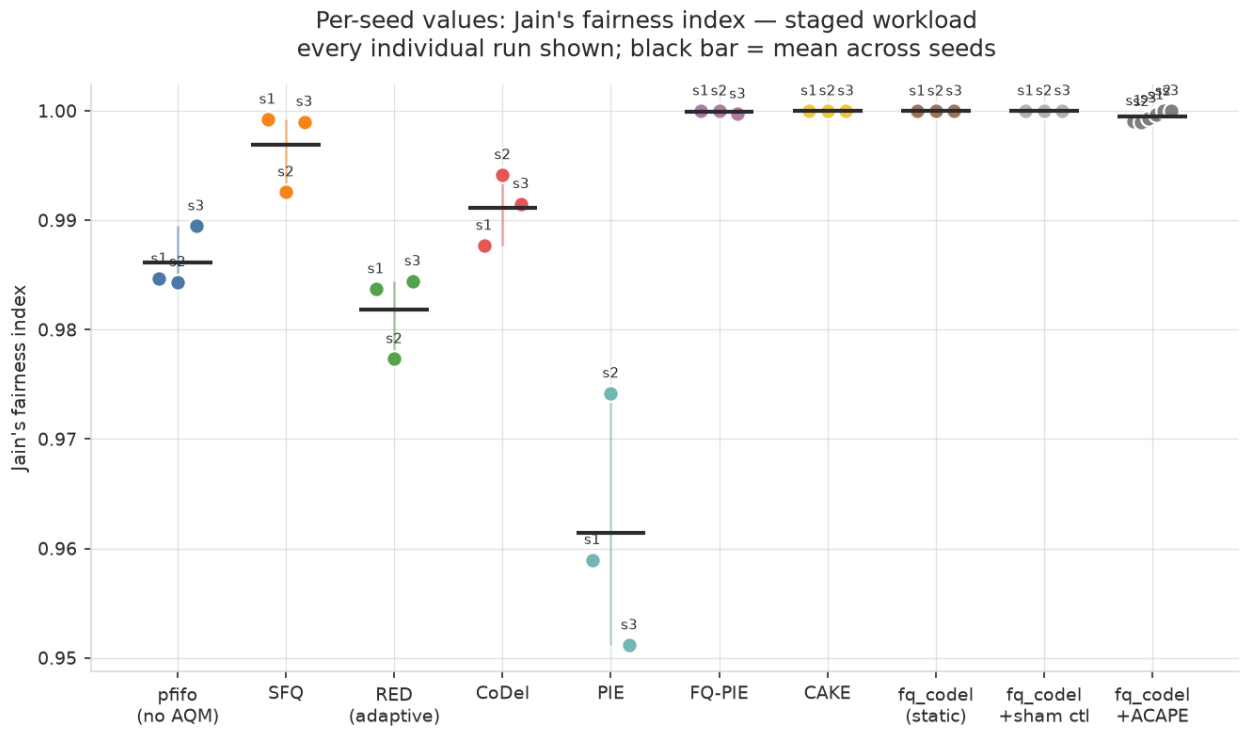


Figure 32: **FIG 21:** fig09_seeds.jain_staged.png. Per-seed values , every individual run shown, mean marked , jain (staged workload)

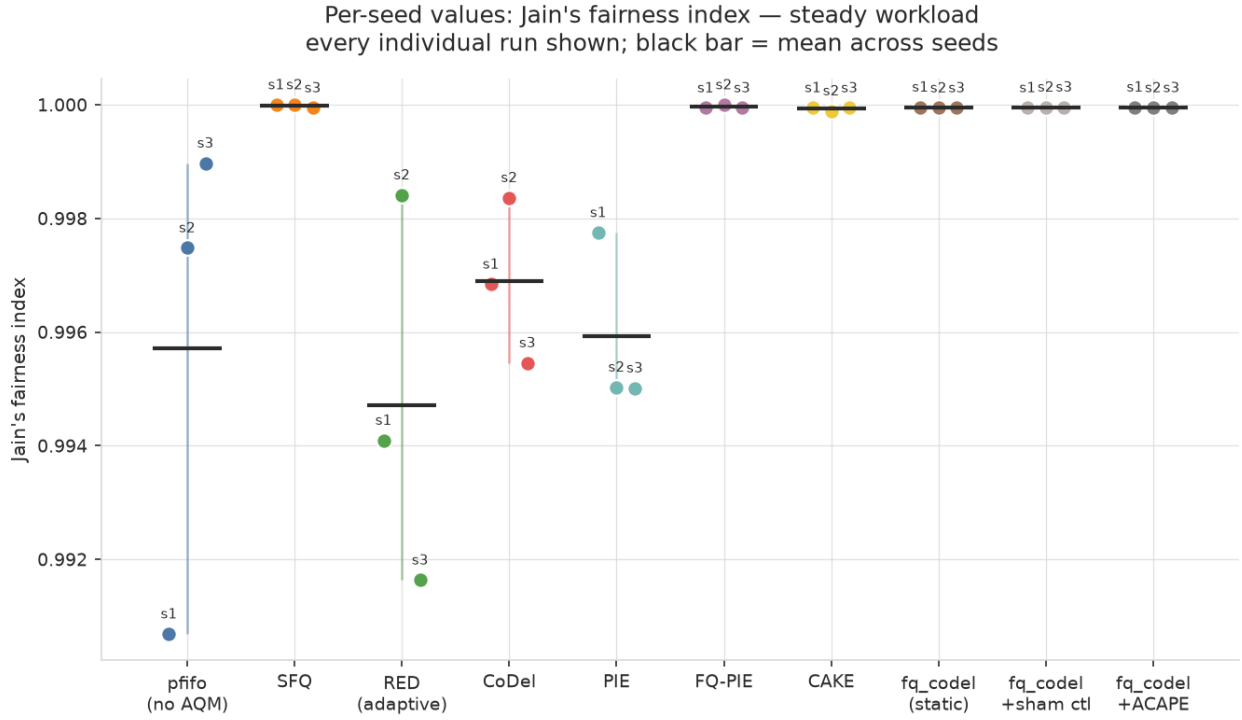


Figure 33: **FIG 22:** fig09_seeds_jain_steady.png. Per-seed values , every individual run shown, mean marked , jain (steady workload)

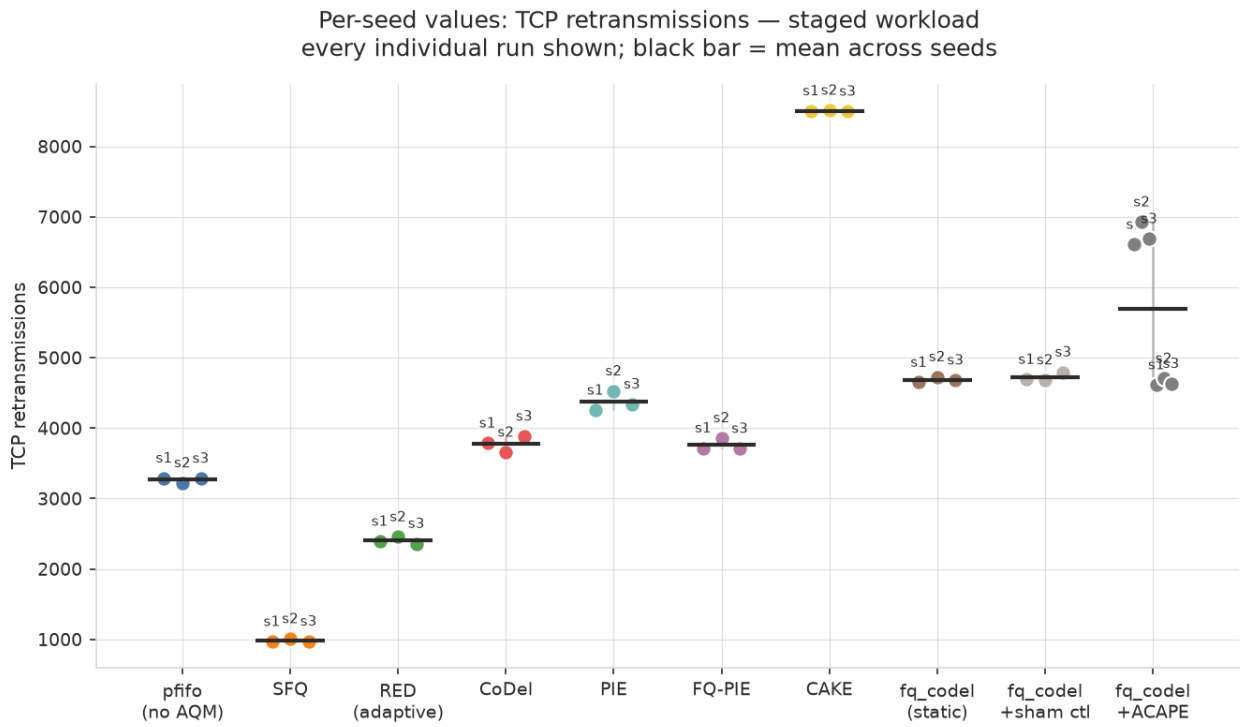


Figure 34: **FIG 23:** fig09_seeds_retransmits_staged.png. Per-seed values , every individual run shown, mean marked , retransmits (staged workload)

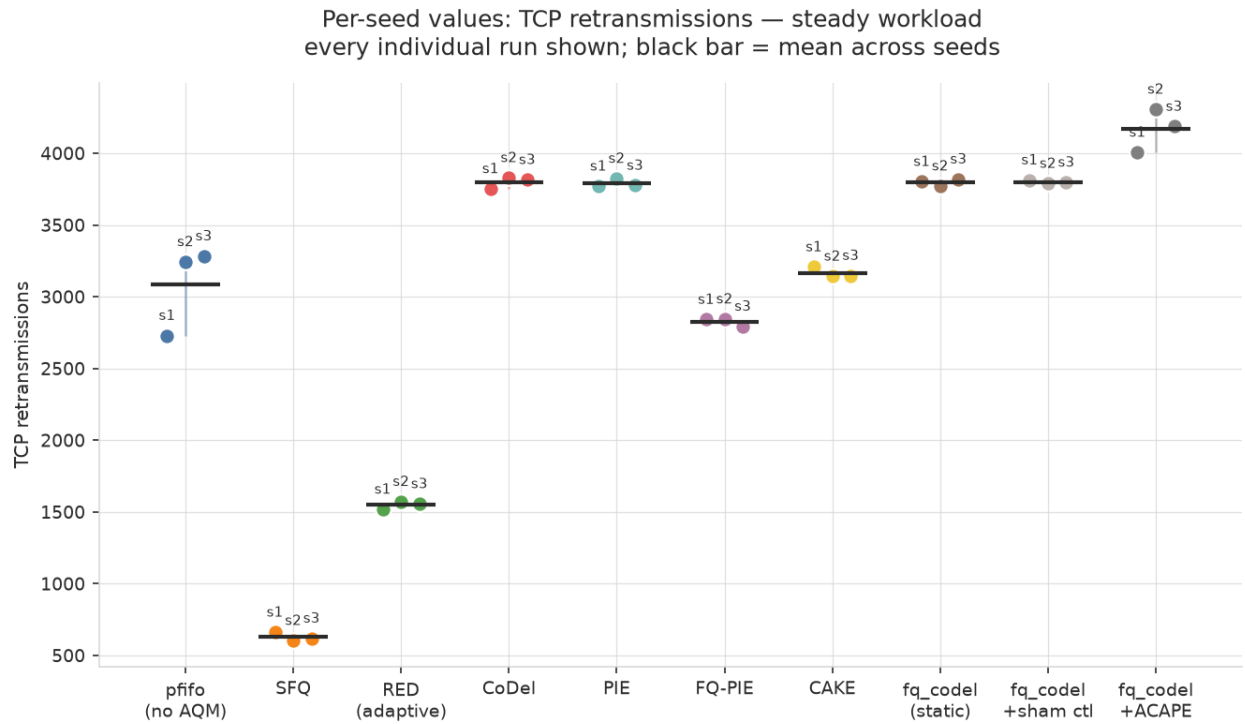


Figure 35: **FIG 24:** fig09_seeds_retransmits_steady.png. Per-seed values , every individual run shown, mean marked , retransmits (steady workload)

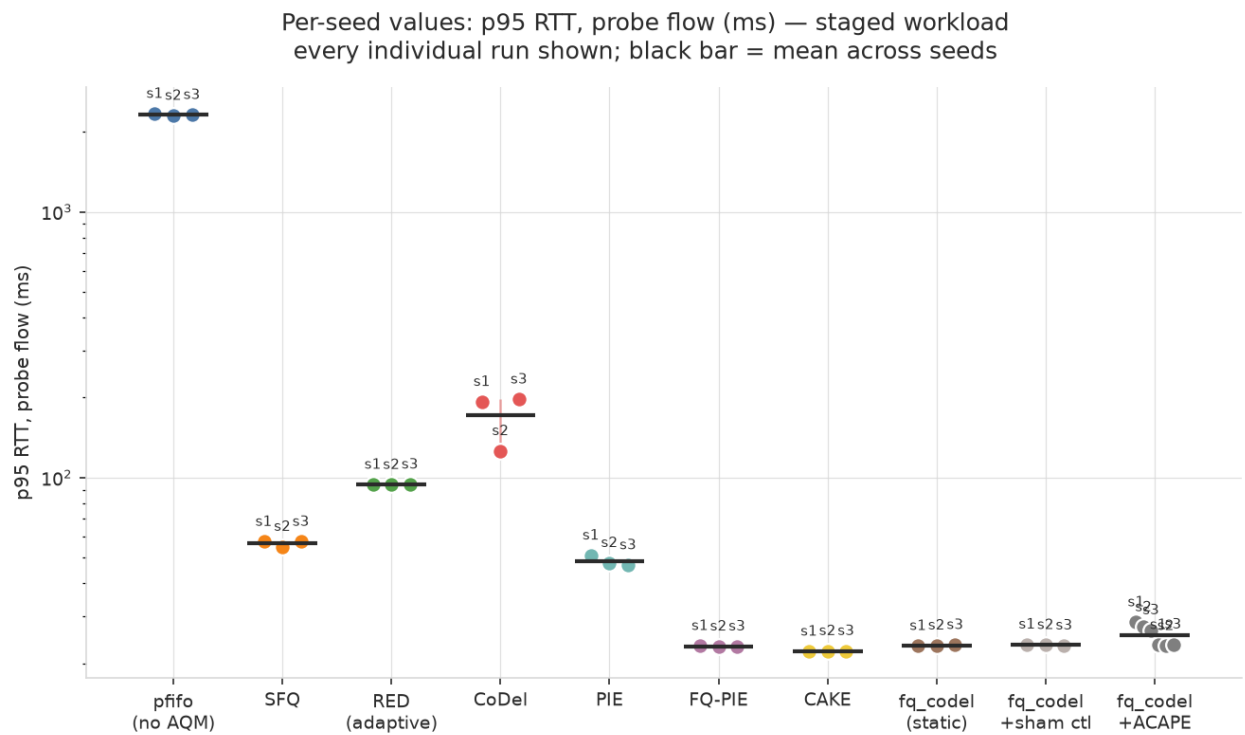


Figure 36: **FIG 25:** fig09_seeds_sparse_rtt_p95_ms_staged.png. Per-seed values , every individual run shown, mean marked , sparse rtt p95 ms (staged workload)

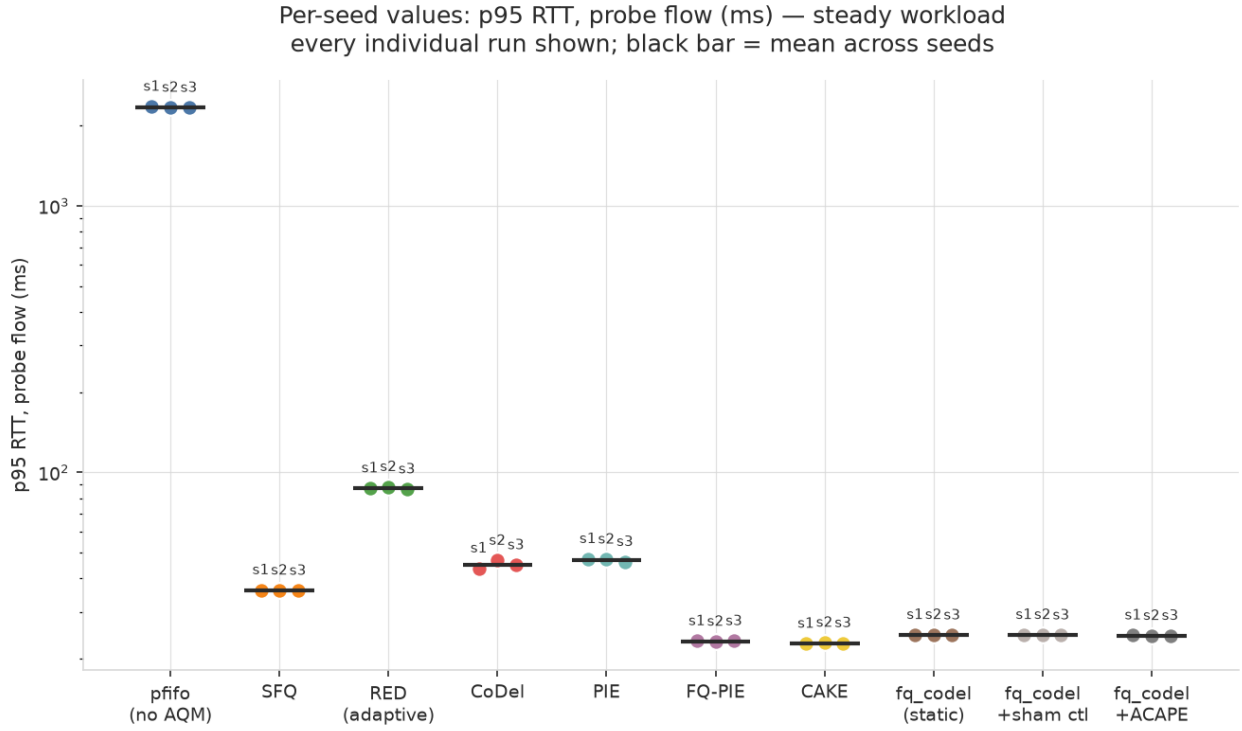


Figure 37: **FIG 26:** fig09_seeds_sparse_rtt_p95_ms_steady.png. Per-seed values , every individual run shown, mean marked , sparse rtt p95 ms (steady workload)

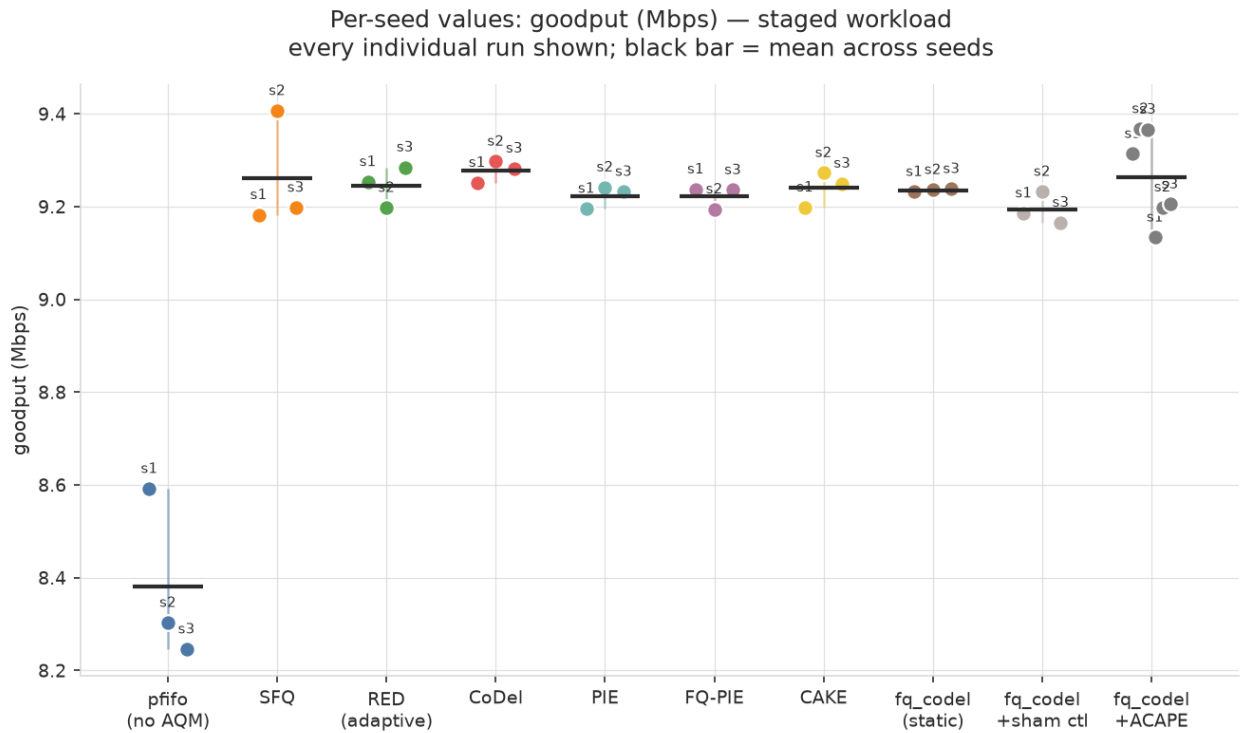


Figure 38: **FIG 27:** fig09_seeds_throughput_mbps_staged.png. Per-seed values , every individual run shown, mean marked , throughput mbps (staged workload)

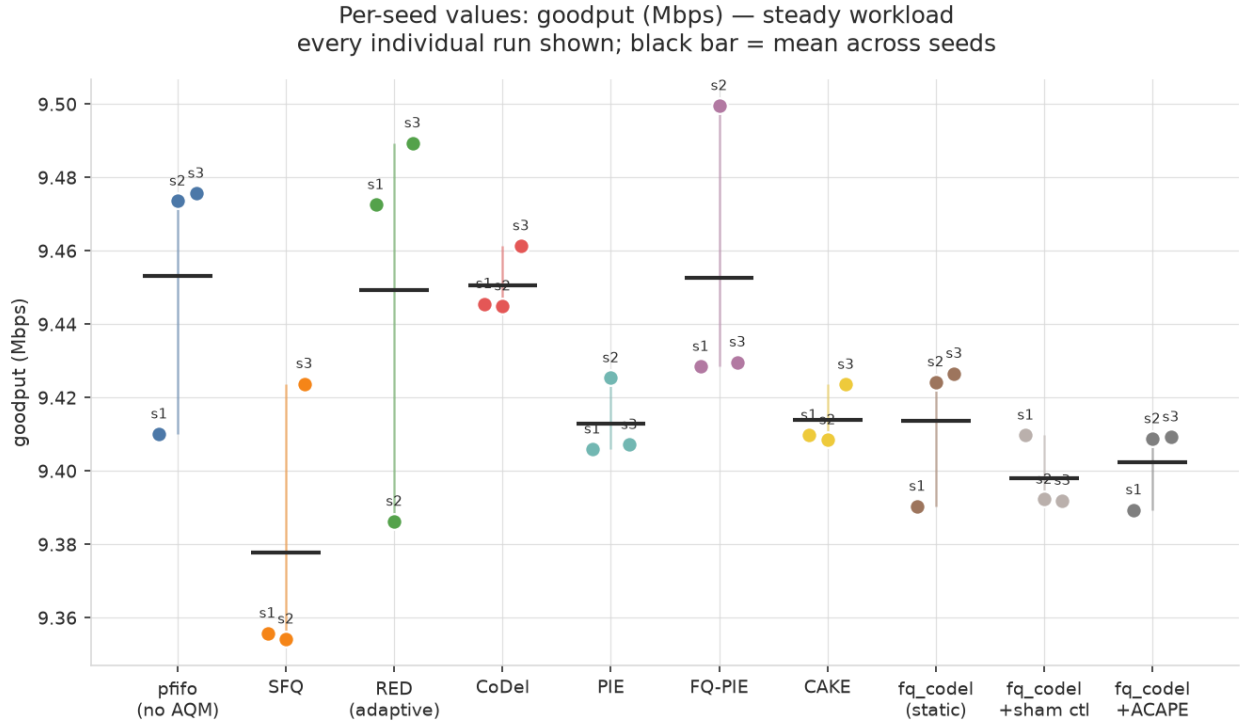


Figure 39: **FIG 28:** fig09_seeds.throughput_mbps_steady.png. Per-seed values , every individual run shown, mean marked , throughput mbps (steady workload)



Figure 40: **FIG 29:** fig10_timeseries_backlog_staged.png. Queue occupancy over time (staged workload)



Figure 41: **FIG 30:** fig10_timeseries_backlog_steady.png. Queue occupancy over time (steady workload)



Figure 42: **FIG 31:** fig11_timeseries_rtt_staged.png. Measured RTT over time (20 Hz probe) (staged workload)

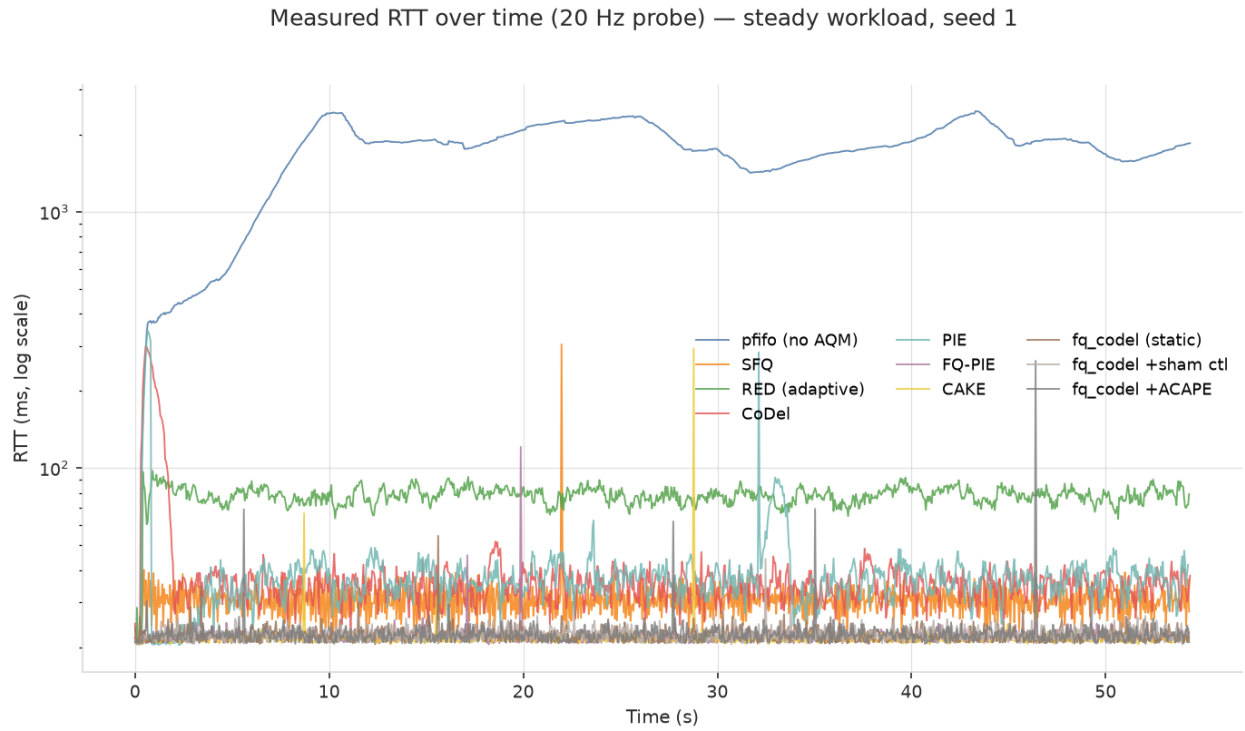


Figure 43: **FIG 32:** fig11_timeseries_rtt_steady.png. Measured RTT over time (20 Hz probe) (steady workload)

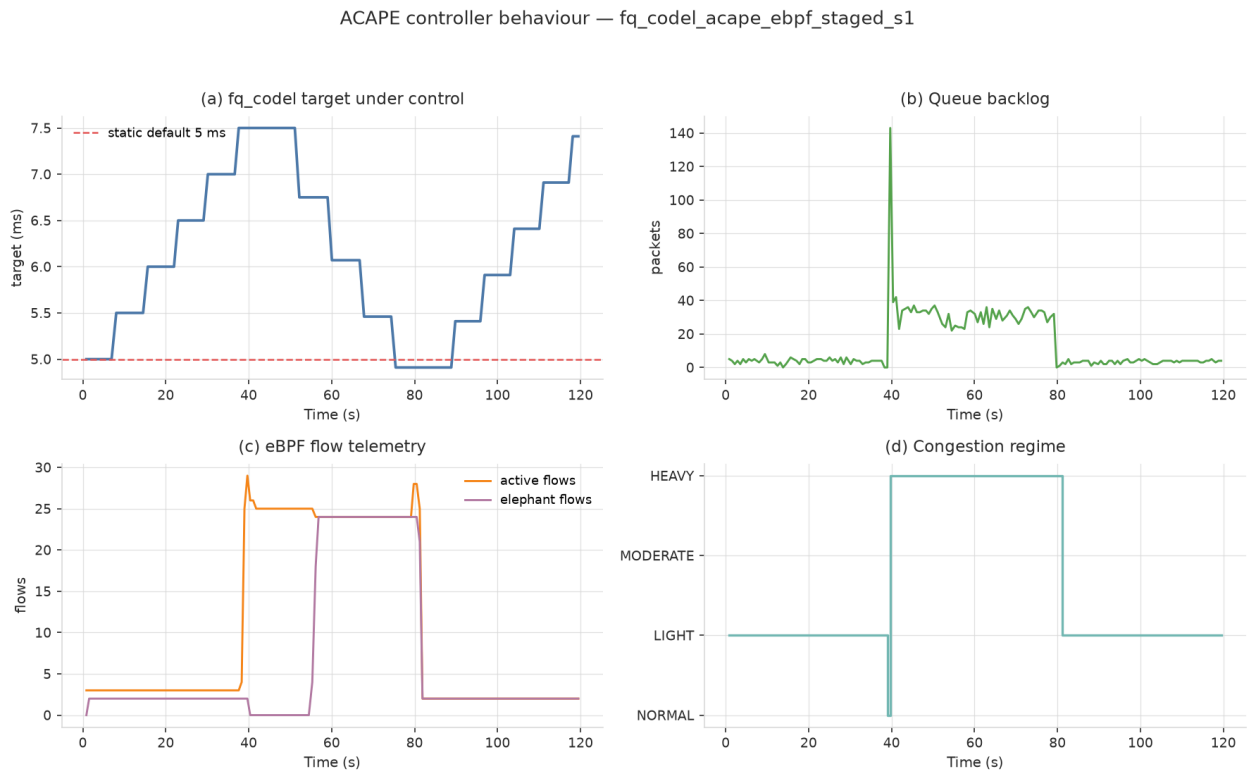


Figure 44: **FIG 33:** fig12_controller_behaviour.png. What the ACAPE controller actually did: target, backlog, eBPF telemetry, regime

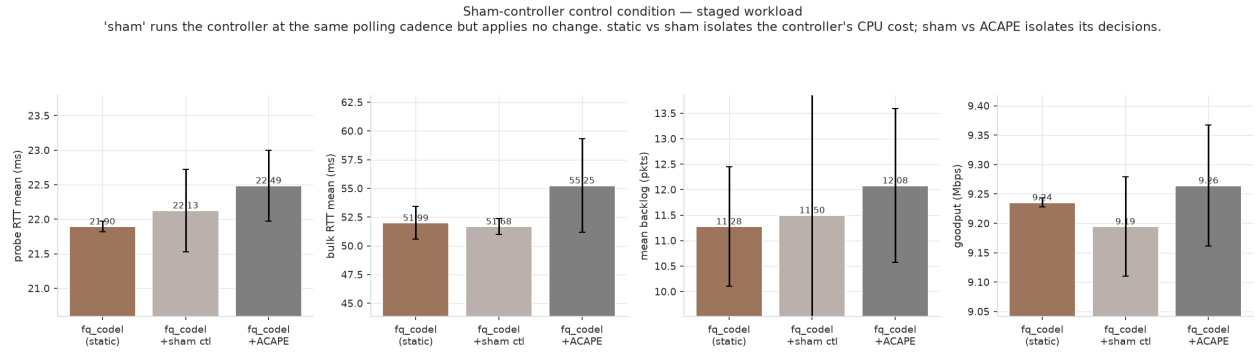


Figure 45: **FIG 34:** fig13_sham_control_staged.png. Sham-controller condition , separates controller CPU cost from control decisions (staged workload)

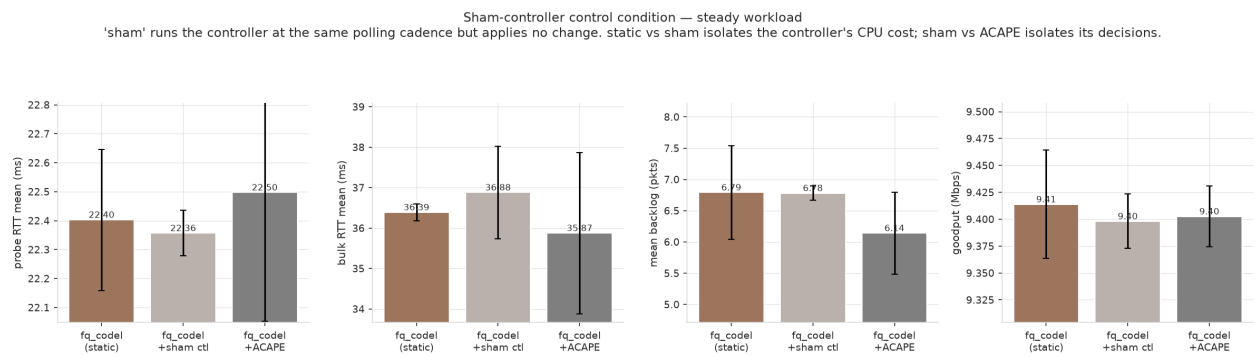


Figure 46: **FIG 35:** fig13_sham_control_steady.png. Sham-controller condition , separates controller CPU cost from control decisions (steady workload)

3 Per-run figures

One figure per experimental run, 63 in total, each showing that run's time series. They are listed here rather than embedded, since the analysis figures above aggregate them and the per-run detail is for checking a specific run rather than for reading through. Every file is in `figures/runs/` in the repository.

ID	Run	Summary
RUN 01	cake_staged_s1	cake, staged workload, seed 1 , goodput 9.1985 Mbps, p95 RTT 22.2 ms, backlog 8.92 pkt
RUN 02	cake_staged_s2	cake, staged workload, seed 2 , goodput 9.2728 Mbps, p95 RTT 22.3 ms, backlog 11.2 pkt
RUN 03	cake_staged_s3	cake, staged workload, seed 3 , goodput 9.25 Mbps, p95 RTT 22.3 ms, backlog 11.28 pkt
RUN 04	cake_steady_s1	cake, steady workload, seed 1 , goodput 9.4098 Mbps, p95 RTT 22.8 ms, backlog 6.89 pkt
RUN 05	cake_steady_s2	cake, steady workload, seed 2 , goodput 9.4085 Mbps, p95 RTT 23.0 ms, backlog 6.55 pkt
RUN 06	cake_steady_s3	cake, steady workload, seed 3 , goodput 9.4236 Mbps, p95 RTT 22.9 ms, backlog 6.58 pkt
RUN 07	code1_staged_s1	code1, staged workload, seed 1 , goodput 9.2517 Mbps, p95 RTT 193.0 ms, backlog 23.58 pkt
RUN 08	code1_staged_s2	code1, staged workload, seed 2 , goodput 9.2975 Mbps, p95 RTT 126.0 ms, backlog 23.71 pkt
RUN 09	code1_staged_s3	code1, staged workload, seed 3 , goodput 9.2825 Mbps, p95 RTT 197.0 ms, backlog 25.09 pkt
RUN 10	code1_steady_s1	code1, steady workload, seed 1 , goodput 9.4456 Mbps, p95 RTT 43.7 ms, backlog 8.98 pkt
RUN 11	code1_steady_s2	code1, steady workload, seed 2 , goodput 9.445 Mbps, p95 RTT 46.7 ms, backlog 8.63 pkt
RUN 12	code1_steady_s3	code1, steady workload, seed 3 , goodput 9.4614 Mbps, p95 RTT 44.9 ms, backlog 8.04 pkt
RUN 13	fq_code1_acape_ebpf_staged_s1	fq.code1 + ACAPE + eBPF, staged workload, seed 1 , goodput None Mbps, p95 RTT 24.7 ms, backlog 11.19 pkt
RUN 14	fq_code1_acape_ebpf_staged_s2	fq.code1 + ACAPE + eBPF, staged workload, seed 2 , goodput None Mbps, p95 RTT 25.3 ms, backlog 11.93 pkt
RUN 15	fq_code1_acape_ebpf_staged_s3	fq.code1 + ACAPE + eBPF, staged workload, seed 3 , goodput None Mbps, p95 RTT 24.5 ms, backlog 12.96 pkt

ID	Run	Summary
RUN 16	fq_codel_acape_staged_s1	fq_codel + ACAPE, staged workload, seed 1 , goodput 9.1339 Mbps, p95 RTT 23.5 ms, backlog 12.5 pkt
RUN 17	fq_codel_acape_staged_s2	fq_codel + ACAPE, staged workload, seed 2 , goodput 9.1986 Mbps, p95 RTT 23.3 ms, backlog 10.4 pkt
RUN 18	fq_codel_acape_staged_s3	fq_codel + ACAPE, staged workload, seed 3 , goodput 9.2051 Mbps, p95 RTT 23.6 ms, backlog 10.16 pkt
RUN 19	fq_codel_acape_steady_s1	fq_codel + ACAPE, steady workload, seed 1 , goodput 9.3894 Mbps, p95 RTT 24.5 ms, backlog 6.34 pkt
RUN 20	fq_codel_acape_steady_s2	fq_codel + ACAPE, steady workload, seed 2 , goodput 9.4088 Mbps, p95 RTT 24.4 ms, backlog 5.84 pkt
RUN 21	fq_codel_acape_steady_s3	fq_codel + ACAPE, steady workload, seed 3 , goodput 9.4093 Mbps, p95 RTT 24.4 ms, backlog 6.24 pkt
RUN 22	fq_codel_sham_staged_s1	fq_codel, staged workload, seed 1 , goodput 9.1858 Mbps, p95 RTT 23.6 ms, backlog 11.93 pkt
RUN 23	fq_codel_sham_staged_s2	fq_codel, staged workload, seed 2 , goodput 9.2323 Mbps, p95 RTT 23.5 ms, backlog 10.3 pkt
RUN 24	fq_codel_sham_staged_s3	fq_codel, staged workload, seed 3 , goodput 9.1657 Mbps, p95 RTT 23.4 ms, backlog 12.26 pkt
RUN 25	fq_codel_sham_steady_s1	fq_codel, steady workload, seed 1 , goodput 9.4098 Mbps, p95 RTT 24.6 ms, backlog 6.82 pkt
RUN 26	fq_codel_sham_steady_s2	fq_codel, steady workload, seed 2 , goodput 9.3923 Mbps, p95 RTT 24.6 ms, backlog 6.73 pkt
RUN 27	fq_codel_sham_steady_s3	fq_codel, steady workload, seed 3 , goodput 9.392 Mbps, p95 RTT 24.5 ms, backlog 6.79 pkt
RUN 28	fq_codel_staged_s1	fq_codel, staged workload, seed 1 , goodput 9.232 Mbps, p95 RTT 23.3 ms, backlog 11.15 pkt
RUN 29	fq_codel_staged_s2	fq_codel, staged workload, seed 2 , goodput 9.2362 Mbps, p95 RTT 23.4 ms, backlog 11.8 pkt
RUN 30	fq_codel_staged_s3	fq_codel, staged workload, seed 3 , goodput 9.2382 Mbps, p95 RTT 23.5 ms, backlog 10.88 pkt
RUN 31	fq_codel_steady_s1	fq_codel, steady workload, seed 1 , goodput 9.3904 Mbps, p95 RTT 24.6 ms, backlog 6.47 pkt
RUN 32	fq_codel_steady_s2	fq_codel, steady workload, seed 2 , goodput 9.4243 Mbps, p95 RTT 24.6 ms, backlog 6.83 pkt

ID	Run	Summary
RUN 33	fq_codel_steady_s3	fq_codel, steady workload, seed 3 , goodput 9.4266 Mbps, p95 RTT 24.5 ms, backlog 7.07 pkt
RUN 34	fq_pie_staged_s1	fq_pie, staged workload, seed 1 , goodput 9.2375 Mbps, p95 RTT 23.3 ms, backlog 18.24 pkt
RUN 35	fq_pie_staged_s2	fq_pie, staged workload, seed 2 , goodput 9.1944 Mbps, p95 RTT 23.2 ms, backlog 18.87 pkt
RUN 36	fq_pie_staged_s3	fq_pie, staged workload, seed 3 , goodput 9.2358 Mbps, p95 RTT 23.2 ms, backlog 18.71 pkt
RUN 37	fq_pie_steady_s1	fq_pie, steady workload, seed 1 , goodput 9.4285 Mbps, p95 RTT 23.3 ms, backlog 11.62 pkt
RUN 38	fq_pie_steady_s2	fq_pie, steady workload, seed 2 , goodput 9.4996 Mbps, p95 RTT 23.2 ms, backlog 12.73 pkt
RUN 39	fq_pie_steady_s3	fq_pie, steady workload, seed 3 , goodput 9.4297 Mbps, p95 RTT 23.3 ms, backlog 11.47 pkt
RUN 40	pfifo_staged_s1	pfifo, staged workload, seed 1 , goodput 8.5918 Mbps, p95 RTT 2339.0 ms, backlog 564.36 pkt
RUN 41	pfifo_staged_s2	pfifo, staged workload, seed 2 , goodput 8.303 Mbps, p95 RTT 2300.0 ms, backlog 558.08 pkt
RUN 42	pfifo_staged_s3	pfifo, staged workload, seed 3 , goodput 8.2458 Mbps, p95 RTT 2315.0 ms, backlog 588.19 pkt
RUN 43	pfifo_steady_s1	pfifo, steady workload, seed 1 , goodput 9.4101 Mbps, p95 RTT 2359.0 ms, backlog 778.62 pkt
RUN 44	pfifo_steady_s2	pfifo, steady workload, seed 2 , goodput 9.4736 Mbps, p95 RTT 2339.0 ms, backlog 793.82 pkt
RUN 45	pfifo_steady_s3	pfifo, steady workload, seed 3 , goodput 9.4757 Mbps, p95 RTT 2334.0 ms, backlog 795.29 pkt
RUN 46	pie_staged_s1	pie, staged workload, seed 1 , goodput 9.1959 Mbps, p95 RTT 51.0 ms, backlog 10.75 pkt
RUN 47	pie_staged_s2	pie, staged workload, seed 2 , goodput 9.2418 Mbps, p95 RTT 47.8 ms, backlog 10.0 pkt
RUN 48	pie_staged_s3	pie, staged workload, seed 3 , goodput 9.2322 Mbps, p95 RTT 46.9 ms, backlog 10.24 pkt
RUN 49	pie_steady_s1	pie, steady workload, seed 1 , goodput 9.406 Mbps, p95 RTT 47.4 ms, backlog 8.7 pkt
RUN 50	pie_steady_s2	pie, steady workload, seed 2 , goodput 9.4254 Mbps, p95 RTT 47.2 ms, backlog 8.77 pkt
RUN 51	pie_steady_s3	pie, steady workload, seed 3 , goodput 9.4072 Mbps, p95 RTT 46.2 ms, backlog 9.05 pkt

ID	Run	Summary
RUN 52	red_staged_s1	red, staged workload, seed 1 , goodput 9.2535 Mbps, p95 RTT 94.2 ms, backlog 21.39 pkt
RUN 53	red_staged_s2	red, staged workload, seed 2 , goodput 9.1977 Mbps, p95 RTT 94.0 ms, backlog 22.31 pkt
RUN 54	red_staged_s3	red, staged workload, seed 3 , goodput 9.2849 Mbps, p95 RTT 94.3 ms, backlog 21.73 pkt
RUN 55	red_steady_s1	red, steady workload, seed 1 , goodput 9.4726 Mbps, p95 RTT 87.3 ms, backlog 26.57 pkt
RUN 56	red_steady_s2	red, steady workload, seed 2 , goodput 9.3863 Mbps, p95 RTT 88.3 ms, backlog 26.63 pkt
RUN 57	red_steady_s3	red, steady workload, seed 3 , goodput 9.4894 Mbps, p95 RTT 86.5 ms, backlog 26.85 pkt
RUN 58	sfq_staged_s1	sfq, staged workload, seed 1 , goodput 9.1821 Mbps, p95 RTT 57.4 ms, backlog 96.93 pkt
RUN 59	sfq_staged_s2	sfq, staged workload, seed 2 , goodput 9.4066 Mbps, p95 RTT 55.0 ms, backlog 92.86 pkt
RUN 60	sfq_staged_s3	sfq, staged workload, seed 3 , goodput 9.1979 Mbps, p95 RTT 57.4 ms, backlog 99.79 pkt
RUN 61	sfq_steady_s1	sfq, steady workload, seed 1 , goodput 9.3558 Mbps, p95 RTT 36.2 ms, backlog 117.58 pkt
RUN 62	sfq_steady_s2	sfq, steady workload, seed 2 , goodput 9.3542 Mbps, p95 RTT 36.1 ms, backlog 115.49 pkt
RUN 63	sfq_steady_s3	sfq, steady workload, seed 3 , goodput 9.4238 Mbps, p95 RTT 36.0 ms, backlog 116.42 pkt

4 Where the numbers come from

Every quantity in the paper and in this report is produced by the analysis pipeline and read into L^AT_EX as a generated macro, so prose cannot drift from data. To regenerate everything from the committed logs:

```
python3 src/analyse.py          # tables, fact macros
python3 src/analyse_law.py      # the scaling law, its table and macros
python3 src/plots.py           # all analysis figures
python3 src/make_index.py       # the figure index this report is built from
python3 paper/make_report.py    # this document
```

The measurement defects found and corrected during the work, including the four that would have decided the scaling-law result rather than measuring it, are documented in the `verification/` directory. The pre-registered hypotheses and their decision thresholds, committed before the data existed, are in `docs/SCALING_LAW.md`, `docs/CROSS_AQM_PREREG.md`, `docs/SERIALIZATION_PREREG.md` and `docs/GATE_PREREG.md`.